

Machining feature and topological relationship recognition based on a multi-task graph neural network

Mingyuan Xia^a, Xianwen Zhao^a, Xiaofeng Hu^{a,b,*}

^a School of Mechanical Engineering, Shanghai Jiao Tong University, Shanghai, China

^b Shanghai Changxing Marine Laboratory, Shanghai, China

ARTICLE INFO

Keywords:

Machining feature recognition
Graph neural network
Multi-task learning
Topological relationship recognition

ABSTRACT

Machining feature recognition (MFR) is crucial for achieving the information interaction between CAD, CAPP, and CAM. It involves reinterpreting design information to obtain manufacturing semantics, which is essential for the integration of product lifecycle information and intelligent process design. The intersection of features can cause geometric discontinuities in 3D models, corrupt single-machining features topologically, and create more complex topological associations. This severely limits the performance of traditional rule-based methods. Learning-based methods can overcome these limitations by learning from data. However, current learning-based methods do not have the capability to identify the topological relationships of machining features, which are crucial for achieving intelligent process planning. To address the issue, this study introduces a new method for machining feature recognition named MFTRNet. The proposed methodology leverages geometric and topological information in B-Rep data to learn three tasks: semantic segmentation, instance grouping, and topological relationship prediction. This allows for instance-level machining feature segmentation and topological relationship recognition simultaneously. Additionally, this paper introduces MFTRCAD, a multi-layer synthetic part dataset that includes feature instance labeling and topological relationship labeling. The dataset comprises over 20,000 3D models in STEP format. MFTRNet is evaluated on MFTRCAD and several open-source datasets at the face-level and feature-level. The experimental results indicate that MFTRNet can effectively achieve instance segmentation of part machining features with accuracy comparable to current cutting-edge methods. Additionally, it has the capability to recognize topological relationships, which compensates for the shortcomings of existing learning-based methods. As a result, this study holds practical significance in advancing the MFR field and achieving intelligent process planning.

1. Introduction

Under new-generation intelligent manufacturing, the relevant systems throughout the entire life cycle of products have been integrated at an unprecedented rate [1]. The transfer of information between product design and manufacturing remains disconnected as two critical phases in the life cycle [2]. The information in the product design stage is stored within the 3D model output by CAD software. It consists of low-level geometric and topological data, such as points, lines, surfaces, and solids. In contrast, CAPP, CAM, and other software in the product manufacturing phase require the input of machining features with semantic information, such as holes, slots, pockets, and chamfers [3]. These systems automate tasks such as process route design and tool machining path planning based on the input part machining features

and topological relationships [4–6]. Machining feature recognition (MFR) is essential in converting design information into manufacturing process information. MFR interprets 3D entities generated by CAD software into machining feature information that contains manufacturing semantics [7]. It plays a key role in achieving the tight integration of information between the design and manufacturing phases of the product life cycle.

After almost 40 years of development, researchers have proposed various MFR methods, mainly categorized into rule-based and learning-based methods [4,8]. Rule-based methods summarize the existence patterns of each machining feature type and formulate matching rules. Depending on the principle of rule formulation and the extraction method, they can be further subdivided into graph-based, volume decomposition, and hint-based methods [9–11]. However, when

* Corresponding author.

E-mail addresses: xmy2000@sjtu.edu.cn (M. Xia), xianwenzhao4711@sjtu.edu.cn (X. Zhao), wshxf@sjtu.edu.cn (X. Hu).

<https://doi.org/10.1016/j.aei.2024.102721>

Received 10 April 2024; Received in revised form 19 June 2024; Accepted 15 July 2024

Available online 26 July 2024

1474-0346/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

multiple features interact, the original regularity of individual features is destroyed, and the complexity of the 3D model is increased due to geometrical discontinuities and changes in topological relationships. Therefore, proposing complete and robust MFR rules for different application scenarios is difficult. In addition, these methods are more time-consuming in rule matching [12].

Learning-based methods employ an end-to-end training process to acquire the mapping between machining features and their corresponding low-level representations, obviating the need for laborious rule formulation. This enables them to effectively capture and handle intricate feature interactions, leveraging a substantial number of training samples [13]. However, current learning-based methods still have some shortcomings, the most important of which is that existing methods cannot recognize topological relationships among features. The topological relationship refers to the spatial configurations between machining features, encompassing adjacency, containment, constraint relationships, and more [14]. These relationships play a crucial role in determining the sequence of machining operations. For instance, if a cylindrical face with a larger diameter is adjacent to another cylindrical face with a smaller diameter, indicating the presence of a common plane, the former must be machined prior to the latter to ensure proper execution [15]. The learning-based method's lack of capability to recognize topological relationships reduces its practical application value, particularly in scenarios such as process planning and toolpath design [16]. Furthermore, learning-based approaches rely on datasets. However, existing datasets for MFR suffer from two obvious shortcomings [13,17–19]. One is that current multi-feature datasets are created by adding features to a single cube part without considering rotational or multi-layer parts. This dataset-creation approach makes it challenging to represent genuine mechanical parts accurately. A multi-layer parts dataset containing cubes and rotators is currently missing. Another drawback is that datasets containing annotations of topological relationships are non-existent.

To tackle the aforementioned challenges, this study introduces MFTRNet, a graph neural network architecture tailored for the recognition of machining features and topological relationships on 3D part models. MFTRNet effectively integrates both geometric and topological information extracted from STEP models, enabling it to simultaneously accomplish three tasks: semantic segmentation, instance grouping, and topological relationship prediction. By automating the extraction of machining features and topological relationships, MFTRNet can support the intelligence and automation of subsequent process planning and other process design tasks. In addition, to train MFTRNet, a multi-layer 3D CAD model dataset containing topological relationship annotations is automatically created by parametric modeling techniques. The key contributions of this paper include the following:

- Based on the 3D model construction script of [18], the first multi-layer part dataset oriented towards machining features and topological relationship recognition is presented. The dataset contains over 20 k 3D CAD models labeled with instance-level machining features and topological relationships in STEP format. This dataset provides researchers with an environment for evaluating MFR methods that are more complex and closer to genuine parts.
- A novel multi-task graph neural network architecture MFTRNet for machining feature recognition is proposed, which efficiently combines geometric and topological information from neutral B-Rep data. The architecture explicitly separates task-specific and shared knowledge and constructs information routing for knowledge fusion. This improves knowledge-sharing efficiency and solves the negative transfer problem exhibited by multi-task learning in MFR.
- The learning task for topological relationship recognition and corresponding prediction methods are proposed. Thus, the connection and mapping between the bottom-level geometric information, the middle-level feature information, and the top-level topological relationship information are realized. The automated extraction of

machining features and their topological relationships lays the foundation for the intelligent planning of subsequent machining processes.

The remainder of this paper is structured as follows: Section 2 reviews existing methods and datasets for MFR. Section 3 describes the design of MFTRNet and MFTRCAD. Section 4 encompasses a comprehensive experimental analysis aimed at substantiating the efficacy of the proposed methodology. Section 5 provides a concise summary of this article while outlining potential avenues for future research.

2. Related work

2.1. Machining feature recognition via deep learning

Deep learning-based object segmentation and classification methods have been extensively utilized in the field of computer vision. These methods can be classified into three categories based on the level of granularity of the image processing: object detection, semantic segmentation, and instance segmentation. Object detection techniques are designed to identify objects within an image and predict their bounding boxes, which indicate the location of the objects [20]. Semantic segmentation techniques are further refined to achieve an accurate understanding of the scene by assigning category labels to each pixel in the image [21]. Instance segmentation techniques further distinguish between instances of objects in each category in an image based on pixel classification [22]. The aforementioned techniques have already demonstrated their practical value in a number of fields, including virtual reality [23], intelligent transportation systems [24], and human movement recognition [25].

The deep learning-based MFR method can be regarded as the migration of computer vision tasks in CAD models. A B-rep model can be regarded as an image, with the points, edges, and faces in the model corresponding to the pixels in the image. Therefore, it is logical to migrate the aforementioned computer vision tasks to part feature recognition. The object detection task determines the types and locations of machining features by predicting the bounding box [26]. The semantic segmentation task categorizes each face in the model and determines the type of feature to which it belongs [18]. Finally, the instance segmentation task groups the constituent faces of the machining feature instances on the basis of semantic segmentation [12].

However, the geometric and topological information embedded in the B-rep model is considerably more complex than the image pixel arrangement. Furthermore, its non-uniform structure makes it challenging to feed directly into the neural network for processing [19]. These challenges make the task of recognizing machining features on parts more difficult. Existing deep learning-based MFR methods can be broadly classified into two categories: two-stage methods and one-stage methods. Moreover, in order to extract information embedded in 3D model files for input to the deep learning network, these methods employ a variety of 3D model representations, including point clouds, voxels, meshes, B-Rep, multi-views, and text in STEP format.

2.1.1. Two-stage methods

Two-stage methods first train a single classifier for machining features and then decompose the multi-feature model based on geometric or topological prior knowledge. Finally, the decomposed sub-models are classified separately to obtain the final result.

The first deep learning method, FeatureNet [13], voxelizes 3D models, similar to the pixel concept of an image, using a 3D convolutional network to identify individual features. The 3D model is then decomposed using a watershed segmentation algorithm. MSVNet [27] is a multi-view approach that uses 2D images projected from the 3D model in different directions for single feature identification and then uses a selective search algorithm for feature localization. Wang et al. [28,29] extracted face types and FAGs from B-Rep and then mixed GNNs and

feature templates for single feature recognition, and then formulated depth-of-cut oriented rules for feature segmentation. Zhang et al. [30] used a two-stage approach to segment intersecting machining features based on attribute adjacency graphs of CAD models. By introducing deep reinforcement learning, this method overcomes the disadvantage that faces can only belong to a single machining feature in traditional deep learning methods. Still, its weak generalization ability and inability to achieve semantic classification of feature instances make it lack practical application value. Yao et al. [31] represented a 3D model with a single feature through a point cloud and utilized PointNet++ to classify the single feature. The point cloud's normal vector and edge concavity are subsequently designed for multi-feature 3D model decomposition. Mesh data is used in [32], where MeshCNN and faster regional convolutional neural networks are used for single feature recognition, and roughness annotations in the MBD model are used for feature localization.

In the above two-stage methods, the decomposition process of the multi-feature model is essentially an artificial rule formulation based on spatial continuity and topological connectivity, which makes it challenging to overcome the problems of spatial separation and topological destruction brought by feature interaction. In addition, there are time-consuming severe defects [12].

2.1.2. One-stage methods

The drawbacks of the two-stage methods, such as the difficulty in handling highly interactive features, the severe time consumption, and the poor real-time performance, make it difficult to practically apply to real CAD models. Therefore, the researchers explored the one-stage methods for MFR to achieve real-time and effective feature recognition of feature-interacting parts by synchronizing the segmentation and classification of features.

With the development of 2D target detection, Shi et al. [26,33] improved two single-stage target detectors, SSD and RefineDet, and successively proposed SsdNet and RDetNet methods in the object detection framework. These methods simultaneously predict the parameters of the 3D axis-aligned bracket box and the classes of machining features on multiple projection views of the 3D model. Finally, the proposals from numerous views are combined to achieve machining feature recognition.

Considering that STEP files hold all the information for 3D modeling, some researchers have analyzed STEP texts for MFR. Yeo et al. [34] extracted attributes of 3D models such as faces, edges, and rings by parsing STEP files and used a multilayer perceptron for semantic segmentation. In particular, Miles et al. [35,36] introduced a natural language processing (NLP) approach for MFR. They analogously designed a STEP parser and successively performed single-feature and multi-feature recognition in an object detection framework based on the encoder-decoder model of the Long Short-Term Memory model (LSTM). However, the lack of explicit 3D model learning leads to a weak connection between geometry and topology.

B-Rep is a well-established, widely used, and clearly expressed representation in 3D solid modeling. It contains all the geometric and topological information necessary for 3D solid reconstruction, involving multi-level attributes such as vertices, edges, faces, rings, and bodies and their association relationships. B-Rep-based methods have been the most prominent methods for MFR and have shown the most competitive ability to achieve 100 % accuracy in single feature recognition [18]. Cao et al. [17] viewed MFR as a semantic segmentation problem and proposed CADNet. It extracts face adjacency graphs from B-Rep and utilizes the parameter vectors of each face as its node representation. However, the fact that the parameter vectors are only applicable to planar surfaces limits its application. Colligan et al. [18] proposed an improved method called Hierarchical CADNet. It discretizes faces into triangular slices and represents them hierarchically by the parameter vectors of these slices and their adjacencies. UV-Net [37] discretizes the faces and edges of the 3D model in its parameter space and learns face and edge embeddings

based on 2D convolution and 1D convolution, respectively.

The instance segmentation framework for MFR has been studied relatively less at present. ASIN [12] utilizes face-level point cloud data to obtain geometric information and then uses two additional PointNet models for semantic segmentation and instance grouping. The final result of instance segmentation is obtained by encoding each face as a 64D vector using a distance matrix and a mean drift clustering algorithm. AAGNet [19] proposes a geometric attribute adjacency graph data structure to address the lost information caused by converting 3D models to intermediate representations such as point clouds and voxels. Instance segmentation of machining features is realized based on the graph data structure using the GNN encoder.

While the one-stage approaches described above accurately achieve machining feature recognition and localization at face-level, they lack consideration of extracting topological relationships between feature instances. This limitation makes it difficult for current MFR technology to support the realization of intelligent machining process planning.

2.2. Datasets for machining feature recognition

Several datasets with single or multiple features in STEP or STL format have been proposed for MFR. Zhang et al. [13] defined 24 standard machining features and proposed the first dataset for MFR. This dataset is a single-feature dataset, and each class contains 6000 3D models stored in STL format. Most of the above two-stage methods are evaluated on this or equivalent reconstructed datasets. Shi et al. [26] provided a multi-feature dataset for object detection with annotations, i.e., parameters of the bounding box and category labels. The samples are saved in STL format and divided into ten groups. Each group samples 1 ~ 10 machining features. Cao et al. [14] proposed the MFCAD dataset, a multi-feature dataset containing 1 ~ 6 planar class features of the features mentioned above, and the samples in MFCAD are stored in STEP format with face-level semantic labels. Colligan et al. [18] improved the generation method of MFCAD and proposed the MFCAD++ dataset. Each sample contains 3 ~ 10 machining features stored in STEP format with face-level semantic labeling. Wu et al. [19] additionally developed the MFInstSeg dataset using the MFCAD++ construction methodology. Each model within the dataset is annotated with labels for face semantic, feature instance, and bottom face.

In summary, there is currently a lack of multi-layer part datasets containing cubes and rotators to represent genuine parts better. And the available datasets of machining features lack annotation of topological relationships.

3. Methodology

3.1. Overview of the proposed approach

In light of the recent progress in learning-based MFR techniques, this study introduces an approach for machining feature recognition and topological relationship extraction leveraging the graph neural network and multi-task learning framework. To facilitate effective training and evaluation of the three tasks, this paper first creates a Machining Feature and Topological Relationship Recognition Dataset called MFTRCAD. This dataset contains over 20 k multi-layered, multi-featured artificial CAD models, each containing instance-level feature labels and topological relationship labels. Then, a novel multi-task machining feature recognition network, MFTRNet, is constructed. Geometric and topological information is extracted from the B-Rep data structure and encoded by a graph neural network. MFTRNet leverages joint representation learning and information routing to efficiently train the three tasks, while simultaneously establishing semantic links between them for accurate recognition of machining features and topological relationships in CAD models. The pipeline of the proposed framework is shown in Fig. 1.

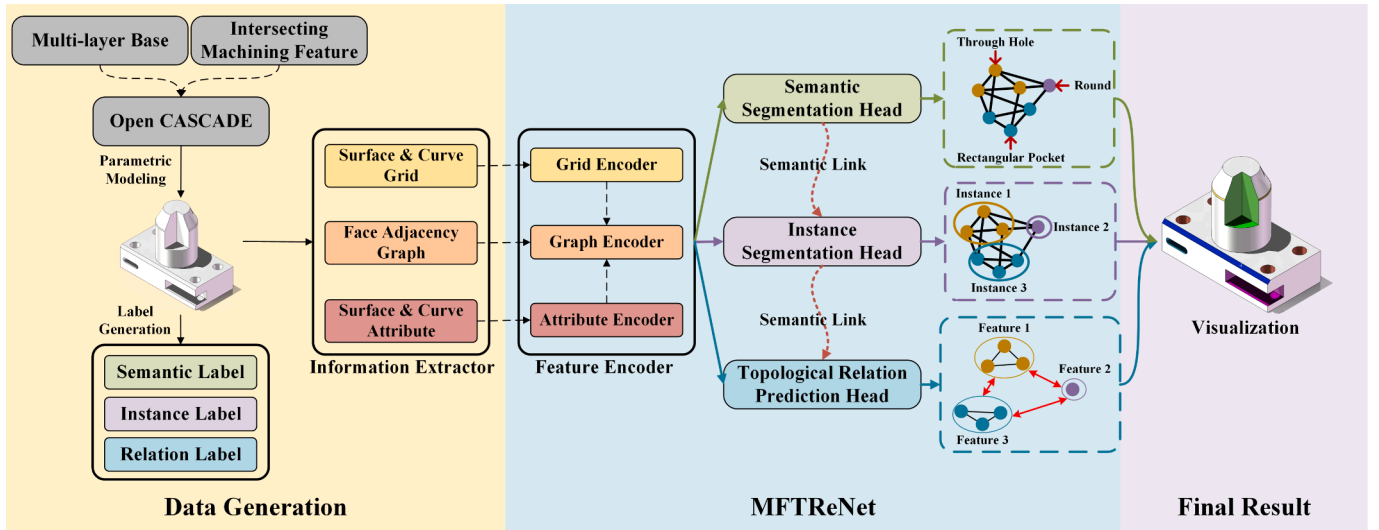


Fig. 1. The pipeline of the proposed framework.

3.2. Dataset creation

As demonstrated in previous research, machine learning techniques necessitate significant quantities of labeled data to facilitate training and evaluation. Nevertheless, there is currently a lack of a dataset that labels machining features and their topological relationships. Therefore, this paper proposes a synthetic dataset of CAD models oriented to machining feature recognition and topological relationship prediction tasks called MFTRCAD.

The generation of MFTRCAD is based on the open-source Python script provided by [18]. MFTRCAD has two improvements over the existing dataset. One is that the part body selected for MFTRCAD contains rotational bodies and is formed into multiple layers by Boolean Fusion operations. Therefore, this dataset is closer to a real 3D part model in production than using only a cube part as a single-layer base. Another point is that each model in this dataset has instance-level feature labels and topological relationship labels, thus providing a data base for the learning task of machining feature topological relationship recognition. In addition, the quality of the generated 3D CAD models is ensured by topological integrity and feature integrity checks. The flow of dataset creation is shown in Fig. 2.

3.2.1. 3D CAD models generation

To automate the generation of large-scale training data, the parametric modeling technique is used to create 3D CAD models using PythonOCC [38], a Python wrapper for Open CASCADE.

A multi-layer part body consisting of a random combination of cubes, cylinders, and cones is first generated, and the smallest bounding cube of this multi-layer part body is created at the same time. Using the same method as in [18] and the 24 classes of machining features defined in [13], 3 ~ 10 features are randomly applied to the bounding cube. Randomly sample and add the topological relationships shown in Table 1 for each machining feature. Finally, Boolean operations are performed to generate features based on the sketches.

Specifically, the decision to add topological relationships and their types is made randomly after sketching the current feature. If topological relationships are to be added, the corresponding sketches are added using the appropriate operations. In order to prevent mutual interference between topological relations, this method employs certain rule constraints during the process of drawing groups of sketches with topological relations. For instance, with regard to the coplanar-based topological relationship (*Coplanar*, *Circle Array*, *Line Array*, and *Mirror*), it is necessary to ascertain whether the feature group with the aforementioned topological relationship already exists on the drawing face and its

parallel faces. If this is the case, it is then necessary to select other faces for sketching. Furthermore, upon the addition of each feature, a systematic examination of the existing features on the sketched face and its parallel faces is conducted. If a coplanar-based topological relationship is identified between the newly added feature and the existing features, additional labels are generated. The relationships defined in Table 1 are mutually exclusive, i.e., there is one and only one relationship between a pair of features. Among them, the three relationships, *Circle Array*, *Line Array*, and *Mirror*, can be considered as special cases of *Coplanar*, i.e., if a set of features is defined as *Coplanar*, they do not have an *Array* or *Mirror* relationship with each other.

When the features on the bounding cube are applied, a Boolean Common operation is performed between the multi-layer part body and the bounding cube. Then, we obtain a multi-layer, multi-feature 3D model. Topological and feature integrity checks are performed on each 3D model. The topological integrity check [19] ensures the validity, closure, and manifoldness of the model. The feature integrity check ensures the model has not corrupted feature instances due to Boolean Common operations. Finally, the tested 3D models and their labels are saved. The distribution of feature types and topological relationship types in the MFTRCAD dataset is shown in Appendix A.

3.2.2. Multi-task label design

For MFTRNet's three learning tasks, specific labeling formats are devised for 3D models, as illustrated in Fig. 3.

The semantic segmentation label marks each face in the 3D model as belonging to the class of its machining feature. For example, the three rectangular through slots in Fig. 3 include 9 faces, so the faces with IDs 18–26 are labeled as that machining feature class. The instance grouping label further classifies the constituent faces of each feature instance into a cluster. When the number of faces is N_f , the instance grouping label can form a 0–1 adjacency matrix of dimension $N_f \times N_f$. In the matrix, elements corresponding to faces belonging to the same feature instance are denoted as 1, whereas the remaining elements are denoted as 0. The topological relationship label treats a feature instance as a node forming a feature-level graph structure, labeling whether there is a topological relationship between nodes and the category of the relationship. As shown in Fig. 3, there is a *Line Array* relationship between the three rectangular through slots, so there are edges typed *Line Array* between the three feature nodes 4,5,6.

3.2.3. Topological and geometric information extraction

The high complexity and irregularity of the B-Rep data structure makes it difficult to be directly processed by machine learning methods

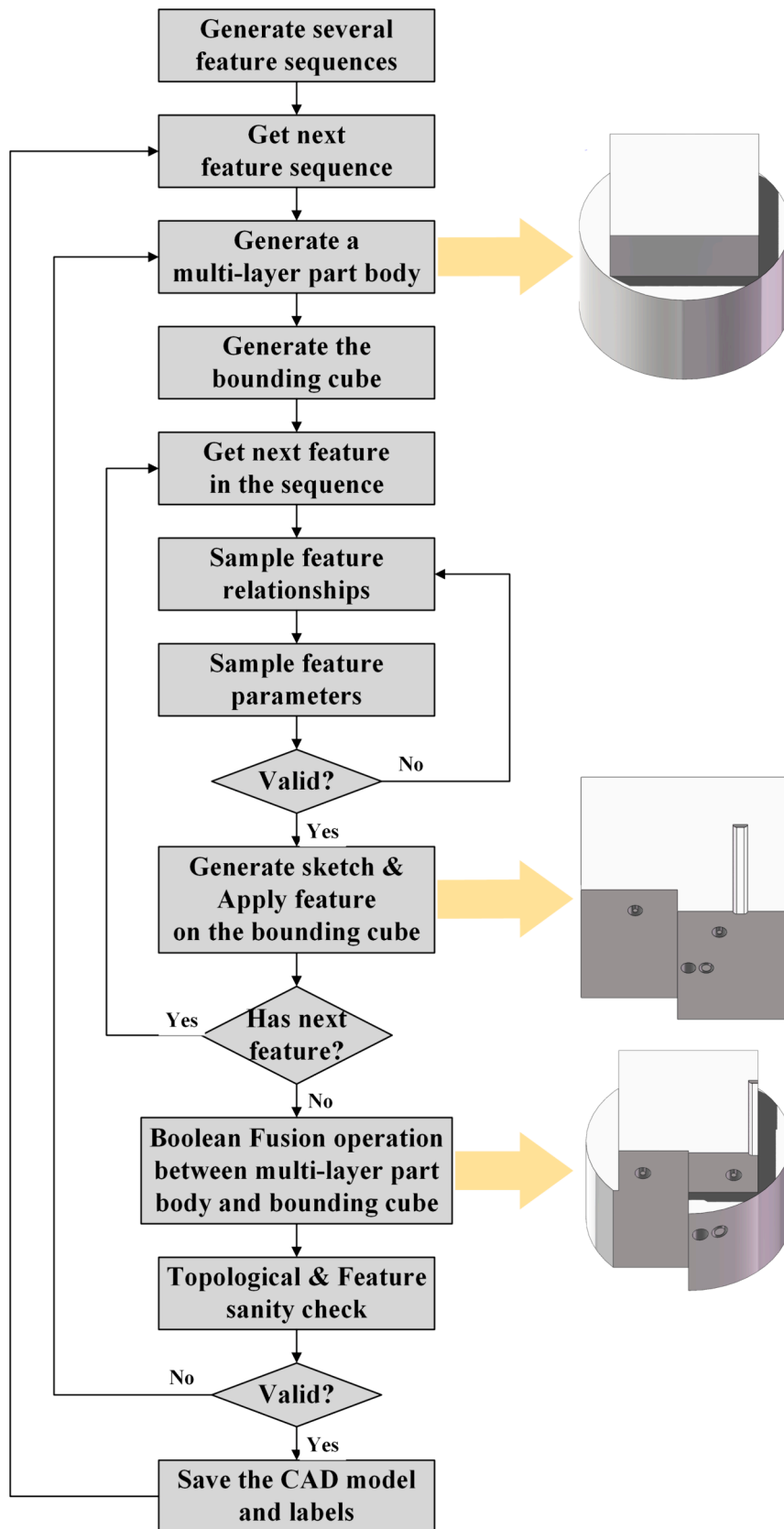


Fig. 2. Flowchart for automated dataset generation.

Table 1
Topological relationship categories and their definitions.

Relationship Type	Definition	Number of related features
Coplanar	Sketches of multiple features are drawn on the same plane of the part body, and the types of features can be different.	≥ 2
Circle Array	Sketches of multiple features are obtained by circular array operations, and the features are of the same type.	≥ 3
Line Array	Sketches of multiple features are obtained by a linear array operation, and the features are of the same type.	≥ 3
Mirror	Sketches of multiple features are obtained by mirroring operations, and the features are of the same type.	$= 2$
Intersecting	Sketches of multiple features with overlap on the same plane	≥ 2
Transition	Edges imposed by rounds and chamfers are part of pre-existing features.	$= 2$
Depend-on	The sketch of the current feature is applied to the face of an existing feature.	$= 2$

[17,18]. Transformation formats such as point clouds, meshes, voxels, etc., are not only limited by resolution but also challenging to capture the full geometric and topological information, which in turn affects the model's performance [37]. Therefore, this paper is based on geometric attribute adjacency graphs proposed in [19] to completely extract the geometric and topological information embedded in the B-Rep data structure.

Geometric information in B-Rep includes entities' size, position, and shape parameters. The faces and edges are defined through parametric equations, which are challenging to process directly by modern machine

learning methods. Jayaraman et al. [37] propose a unified representation for B-Rep data that utilizes U and V parameter domains of faces and edges to model geometry. In this paper, based on the same method, the faces and edges are discretized as UV grids with a fixed step size. The geometric information is characterized using each grid point's point coordinates, normal vectors, and tangent vectors. In addition, the geometric information extracted in this paper contains some attributes of faces and edges. The face attributes include face type, area, and center of mass coordinates; the edge attributes include edge type, length, and concavity.

Topological information in B-Rep refers to the internal connectivity between entities. This topological information can be conveniently and efficiently represented using a graph data structure known as a face adjacency graph (FAG). In the FAG, the nodes correspond to the faces of the 3D model, while the connectors represent the edges. When two faces are adjacent in the B-Rep model through an edge, a corresponding connector is established between the respective nodes in the FAG. In this paper, the FAG is constructed from B-Rep models through the PythonOCC API and stored as an undirected graph data structure that can be processed with the Pytorch Geometric Library (PyG) [39]. The features of the nodes in the graph include face grid point parameters and face attributes. The features of the connectors in the graph include the edge grid point parameters and the edge attributes. The features of the graph data and their dimensions are shown in Table 2.

3.3. MFTRNet: Machining feature and topological relationship recognition network

Graph Neural Networks (GNNs) are a powerful deep learning technique that enables efficient processing of graph-structured data [40,41].

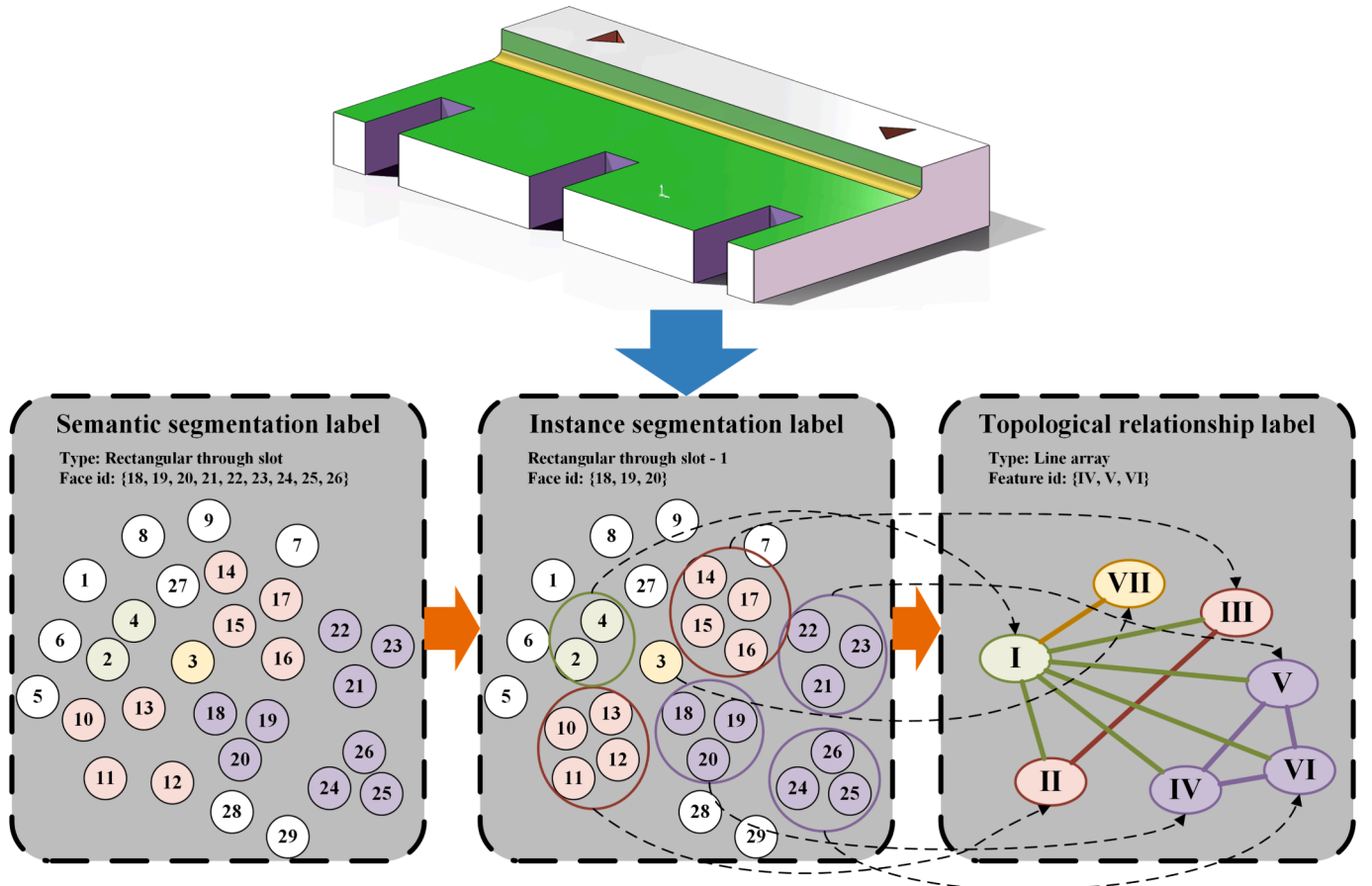


Fig. 3. An example of label design in MFTRCAD.

Table 2
Geometric features of faces and edges.

Element	Feature	Definition	Dimension
Face	Type	One-hot code characterizing the geometric type of the face (plane, cylinder, cone, sphere, torus, revolution, extrusion, offset, or other)	9D
	Area	The area of the face	1D
	Centroid	The centroid coordinates of the face	3D
	Rational	Is a rational B-spline surface	1D
	UV Grid	Coordinates(3D), normal vectors(3D) and visibility(1D) of each grid point	7D
Edge	Type	One-hot code characterizing the geometric type of the edge (circular, closed, elliptical, straight, hyperbolic, parabolic, Bezier, non-rational B-Spline, rational B-Spline, offset, or other)	11D
	Length	The length of the edge	1D
	Convexity	One-hot code characterizing the convexity of the edge (concave, convex, or smooth)	3D
	U Grid	Coordinates(3D), tangent(3D), normal vectors to the left neighboring surface(3D) and normal vectors to the right neighboring surface(3D) for each grid point	12D
	V Grid	Coordinates(3D), tangent(3D), normal vectors to the left neighboring surface(3D) and normal vectors to the right neighboring surface(3D) for each grid point	12D

Multi-task Learning (MTL) enhances the model's generalization ability and learning efficiency by utilizing the underlying similarities between independent or interdependent tasks [42]. 3D models can be naturally abstracted to graph data structures, and the machining feature recognition can be decomposed into multiple interrelated learning tasks such as semantic segmentation and instance localization. Based on the progress of research on graph neural networks and multi-task learning paradigms, a novel network architecture for Machining Features and Topological Relationship Recognition called MFTRNet is proposed. MFTRNet combines geometric and topological information in neutral B-Rep data to achieve machining feature recognition, localization, and topological relationship extraction. As shown in Fig. 4, MFTRNet consists of three main components: a geometric information encoder, a topological information encoder, and a multi-task head.

The geometric information encoder is used to encode the UV grid data and geometric attribute data in B-Rep. The feature vectors are extracted from the grid data of faces and edges using Convolutional Neural Network (CNN), and the geometric attribute data are embedded using Multilayer Perceptron (MLP). Then the grid embeddings corresponding to the faces and edges are concatenated with the geometric attribute embeddings as the node and edge feature vectors of the FAG, respectively, to form a graph data structure for input into the topological information encoder. The topological information encoder processes the input graph data through a set of GNN encoders to extract high-level knowledge for subsequent tasks. This includes both multi-task shared encoders and task-specific encoders. For each task, information routing is constructed through a gating network, which fuses the shared

knowledge with the task-specific knowledge to obtain the encoded information needed to accomplish the specific task. The negative transfer problem and seesaw phenomenon during multi-task learning can thus be avoided [43].

Finally, the encoded information associated with each task is fed into its respective task head, comprising the semantic segmentation head, instance grouping head, and topological relationship prediction head. The semantic segmentation head computes for each face a probability that the face belongs to the machining feature class. The instance grouping head predicts the probability that each pair of faces belongs to the same feature instance. The relationship prediction head aggregates feature instance face information for link prediction in the feature dimension and classifies links between features. Semantic links exist between the three task heads, thus enabling cross-task knowledge transfer and increasing model robustness.

3.3.1. Geometric information encoder

The architecture of the geometric information encoder is shown in Fig. 5. The inputs to the geometric information encoder include the face grid data shaped $N_f \times (N_u \times N_v \times 7D)$, the edge grid data shaped $N_e \times N_u \times 12D$, the face geometric attributes shaped $N_f \times 14D$, and the edge geometric attributes shaped $N_e \times 15D$. N_f is the number of faces and N_e is the number of edges. N_u and N_v are the numbers of sampling points in the U and V parameter domains. The geometric information mentioned above is fed into the corresponding encoders. The face grid encoder comprises three 2D CNN layers, an average pooling layer, and a flatten layer, which embed the face grid information into 32D vectors. The edge grid encoder structure is the same as the face grid encoder, except that the 2D CNN layer is changed to a 1D CNN layer, and the edge grid is also embedded as a 32D vector. The encoders for face attributes and edge attributes are composed of MLPs, which embed the attribute information as 32D vectors. The embeddings of faces and edges are then concatenated separately to obtain 64D feature vectors that describe the geometric information of faces and edges. The graph data is constructed using the FAG. The embeddings of faces and edges are mapped to the features of graph nodes and connectors, respectively.

3.3.2. Topological information encoder

The topological information encoder processes the graph data obtained above through a series of GNN blocks, fusing topological and geometric information to obtain high-level feature embeddings for multi-task learning. The architecture of the GNN block is shown in Fig. 6. The graph data is fed into several graph encoders consisting of a graph convolution layer, an activation layer, and a graph normalization layer. In the graph convolutional layer, we have chosen the method proposed in [44] to aggregate and update the information of the graph. It combines residual connections and gating mechanisms to improve the performance and robustness of GNN. The graph convolutional layer can be expressed as follows:

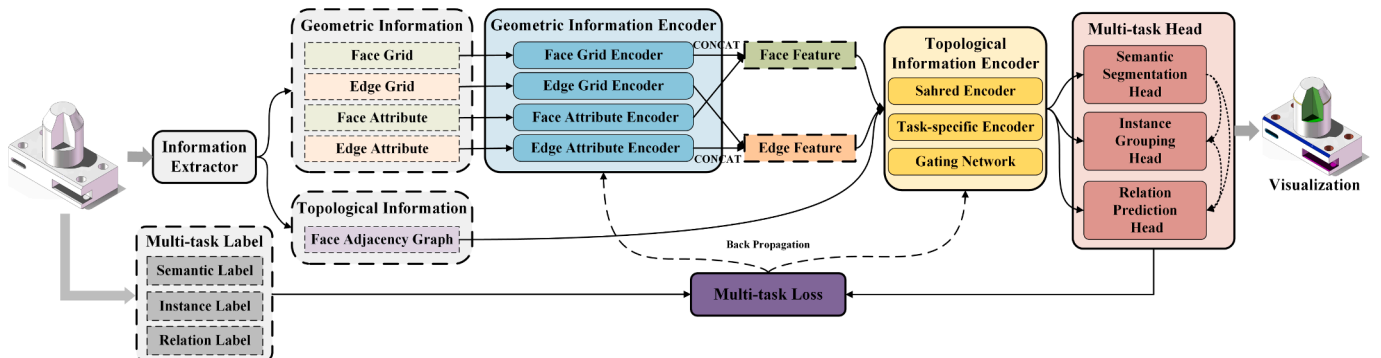


Fig. 4. The architecture of MFTRNet.

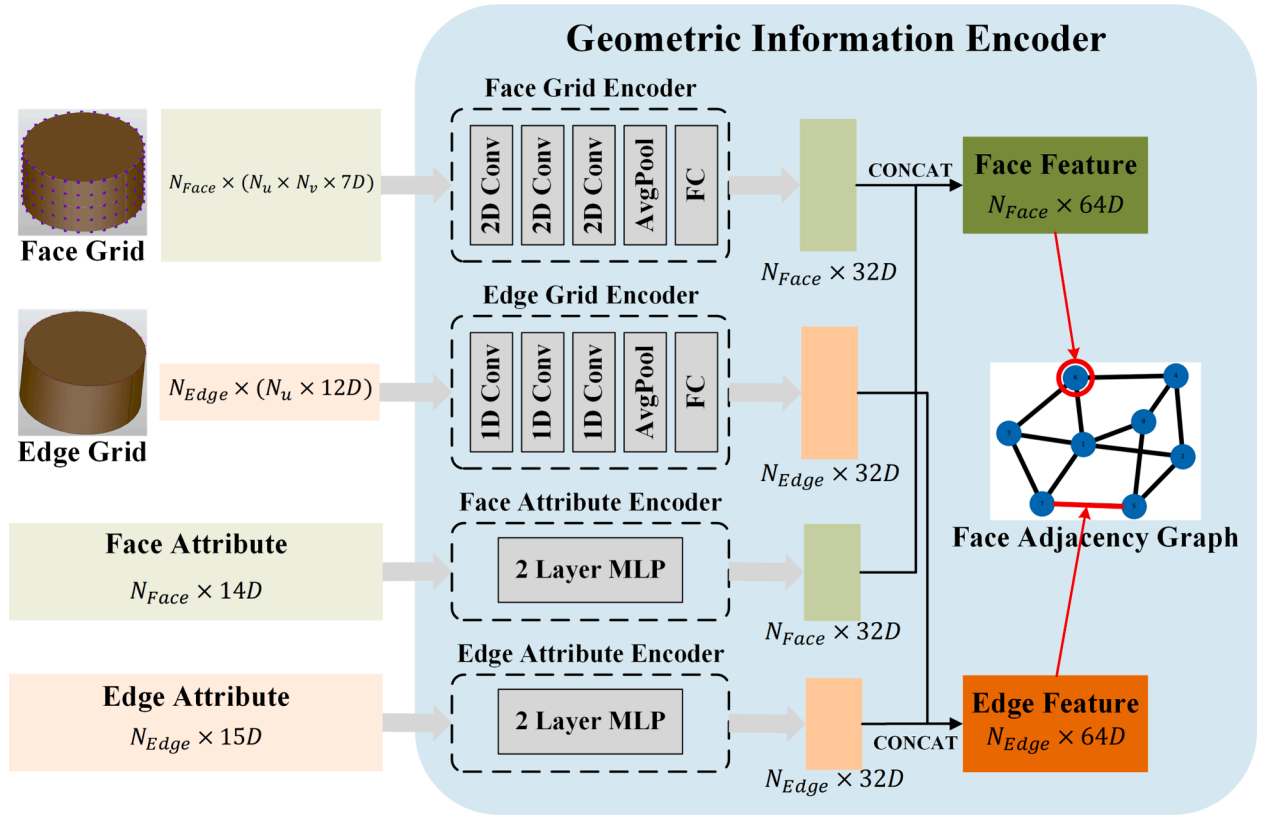


Fig. 5. The architecture of geometric information encoder.

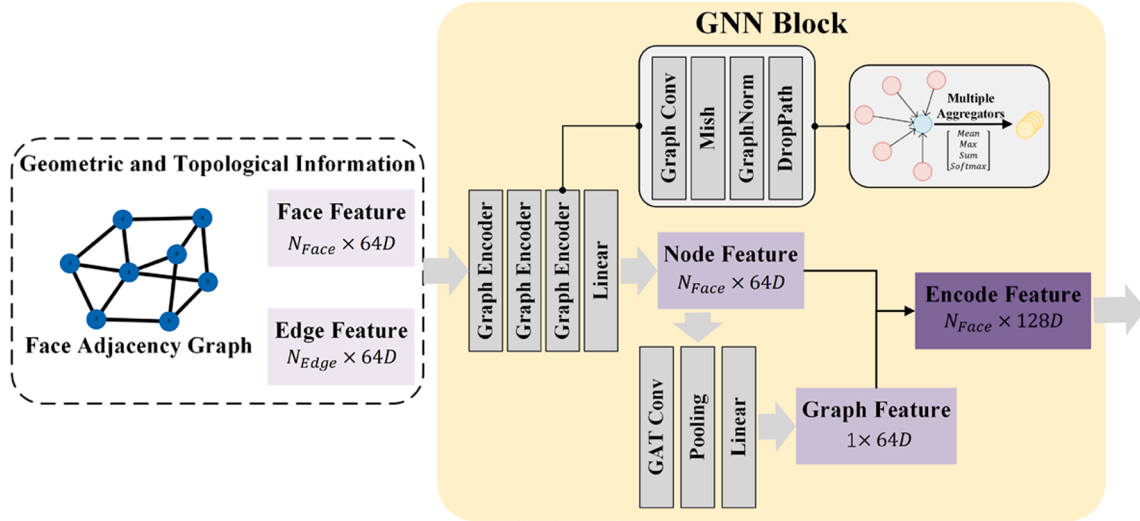


Fig. 6. The architecture of the GNN Block.

$$\mathbf{x}'_i = \mathbf{W}_1 \mathbf{x}_i + \sum_{j \in \mathcal{N}(i)} \eta_{ij} \odot \mathbf{A}(\mathbf{x}_i, \mathbf{x}_j, \mathbf{e}_{ij}) \quad (1)$$

$$\eta_{ij} = \sigma(\mathbf{W}_2 \mathbf{x}_i + \mathbf{W}_3 \mathbf{x}_j) \quad (2)$$

where $\mathbf{x}_i$ and $\mathbf{x}'_i$ represent the features of node i before and after the update, respectively. $\mathcal{N}(i)$ represents the set of neighboring nodes of node i . $\mathbf{e}_{ij}$ denotes the edge between nodes i and j . $\mathbf{A}$ is the aggregation function. $\mathbf{W}_1, \mathbf{W}_2, \mathbf{W}_3$ are the learnable weight parameters, and σ denoting the Sigmoid function. The choice of aggregation function $\mathbf{A}$ during message passing between neighboring nodes has a decisive

impact on the performance of GNN. The experimental results of the study [45] have indicated that the learning effect of the graph network can be further enhanced by the use of multiple aggregators. Therefore, the GNN block uses the same four aggregation functions as in the literature: *Mean*, *Std*, *Max*, and *Min*.

The feature vector shaped $N_f \times 64D$ of each node is obtained after learning through multiple graph encoders. Further, the node feature vectors are passed into a readout layer to get a graph global information embedding shaped $1 \times 64D$. The readout layer consists of a GAT layer [46], a pooling layer, and a linear layer. Finally, the graph global information embedding is concatenated with the feature vectors of each

node to obtain the embedding shaped $N_f \times 128D$ that characterizes all the information of the graph.

Conventional multi-task learning frameworks tend to adopt the “Hard Parameter Sharing” paradigm, i.e., embedding the input information with a shared encoder and then using the embedding for several different tasks. However, the correlation patterns of the three tasks defined in this paper are complex and strongly dependent on the sample, which results in a model that improves some tasks but hinders the performance of others. This makes multi-task learning less effective than single-task models, which is known as the seesaw phenomenon [43]. Specifically, the semantic segmentation task is concerned with categorizing the different faces of a part according to their semantic categories; the instance grouping task requires identifying and distinguishing between different instances of the same semantic category; and the topological relationship prediction task aims to understand the spatial relationships between different instances. Although these three tasks are related in MFR, they differ in their goals and the type of information to be recognized. Therefore, in the case of shared resources, the performance improvement of one task may take up more model resources, thus affecting the performance of the other tasks. In addition, due to the uneven sample distribution of the three tasks in the dataset, the model may tend to optimize those tasks that are easier or more frequently occurring while ignoring the others.

To cope with the seesaw phenomenon during model training, the topological information encoder of MFTReNet adopts the architecture shown in Fig. 7. The topological information encoder comprises four encoding modules, three of which are task-specific (corresponding to the subsequent three tasks, respectively) and one multitask-shared encoding module. Each encoding module is comprised of multiple stacked GNN blocks (Fig. 6), with the number of GNN blocks in each module being a hyper-parameter that can be tuned. Each encoding module outputs an embedding shaped $N_f \times N_b \times 128D$, and N_b is the number of GNN blocks in each encoding module. In particular, the shared encoding module in the topological information encoder is responsible for learning the shared knowledge of multiple tasks, while the task-specific knowledge is extracted by the respective task-specific encoding module. Each subsequent task head absorbs both the shared encoding knowledge and its own task-specific encoding knowledge. This implies that the parameters of the shared encoding module are updated in response to the collective influence of all tasks, whereas the parameters of the task-specific encoding module are updated solely in response to the influence of the respective task.

In the topological information encoder, shared and task-specific knowledge are selectively fused through an information route constructed from a gating network. The gating network uses the input graph

data as a selector and computes the weights of the knowledge vectors through a network consisting of a GCN layer [47], a Linear layer, and a SoftMax layer. The weighted sum shaped $N_f \times 128D$ of the outputs of the shared encoding module and the task-specific encoding module is computed based on the weight output from the gated network, thereby enabling the efficient fusion of shared and task-specific knowledge.

The embedding for task k is expressed more precisely as follows:

$$g^k(x) = w^k(x)S^k(x) \quad (3)$$

where x is the input data, and $w^k(x)$ is the gating network used to calculate the weight vector of task k :

$$w^k(x) = \text{SoftMax}(F(x)) \quad (4)$$

where F denotes the transformation performed on the input by the GCN layer and the linear layer. $S^k(x)$ is a matrix composed of shared embeddings and task k 's specific embeddings:

$$S^k(x) = [E_{(k,1)}^T, E_{(k,2)}^T, \dots, E_{(k,m)}^T, E_{(s,1)}^T, E_{(s,2)}^T, \dots, E_{(s,n)}^T]^T \quad (5)$$

where $E_{(k,i)}$ represents the output of each GNN block in the task-specific encoder and m is the total number. $E_{(s,i)}$ represents the output of each GNN block in the shared encoder and n is the total number.

The proposed architecture effectively mitigates the harmful parameter interference between shared and task-specific knowledge by explicitly separating shared and task-specific knowledge. This allows different types of encoding modules to focus on learning different knowledge efficiently without interference. Combined with the advantage of the gating network's dynamic fusion of representations based on inputs, it achieves a more flexible balance between tasks and better handles task conflicts and sample-dependent correlations. Furthermore, the proposed architecture maintains the advantages of the multi-task learning paradigm, including strong generalization ability and data efficiency.

3.3.3. Multi-task head

Each task-specific feature vector shaped $N_f \times 128D$ is obtained through the topological information encoder and input into the corresponding task head, as shown in Fig. 8.

The semantic segmentation head consists of an MLP that classifies each node based on the input feature vectors, thus determining the machining feature type to which each face of the model belongs. The output is a probability vector shaped $N_f \times N_c$ indicating the confidence level of each face's machining feature type, where N_c is the number of machining feature categories. Considering that there is a particular

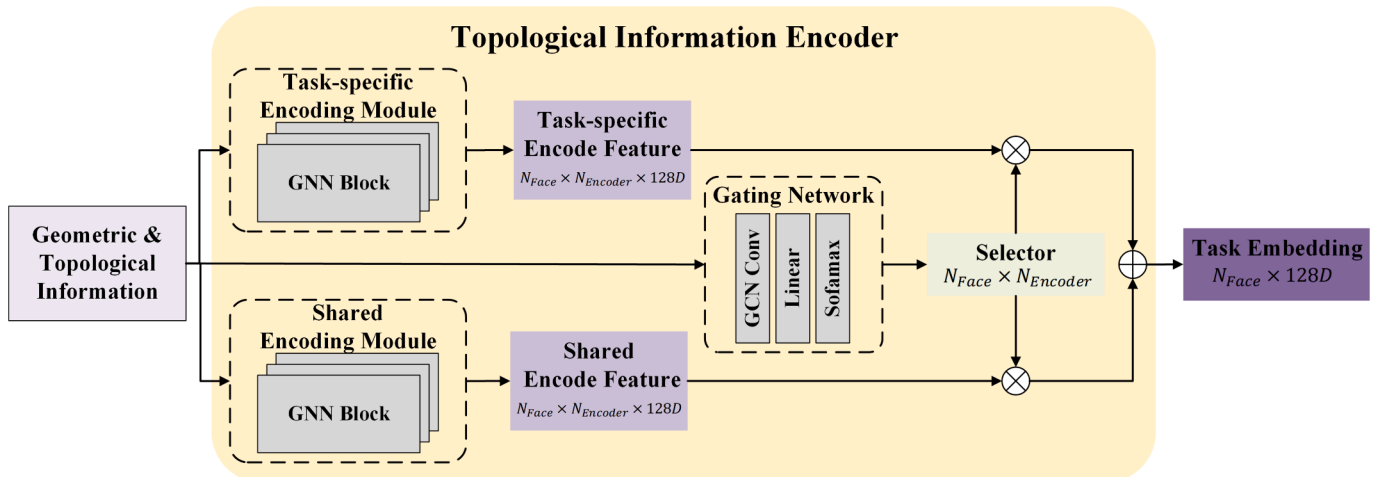


Fig. 7. The architecture of topological information encoder.

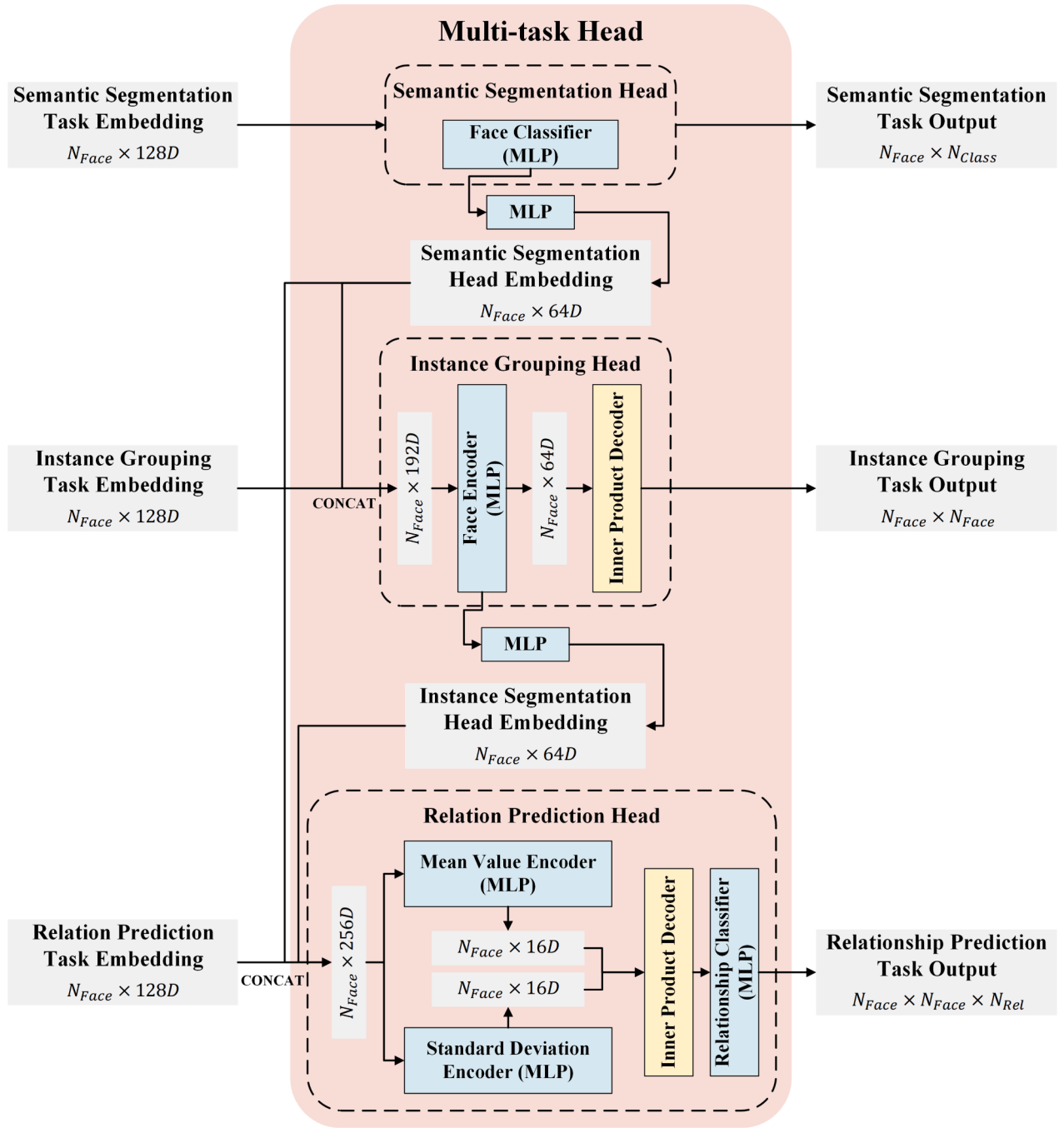


Fig. 8. The architecture of multi-task head.

imbalance in the amount of individual machining features in the dataset proposed in this paper, Focal Loss [48] is chosen as the loss function in the training process, which is expressed as:

$$L_{Sem} = -\alpha_t(1 - p_t)^\gamma \log(p_t) \quad (6)$$

where p_t is the probability vector of the semantic segmentation head output. α_t and γ are the weight coefficients that can be adjusted. Compared with the cross-entropy loss, Focal Loss weights the classification scores of the model output into the loss function, which increases the loss weight of the difficult samples and is suitable for dealing with the classification problem with unbalanced samples. The cross-entropy loss function is used for the MFCAD++ and MFInstSeg datasets, which do not have category imbalance problems.

The instance grouping head consists of an MLP-based face encoder

and an inner product decoder. The embedding of the input linear classification layer in the semantic segmentation head is fed into the instance grouping head through an MLP semantic transformation layer to establish a semantic link between the two heads. Thus, the instance grouping head input is a feature vector shaped 192D concatenated from the task-specific feature vector and the semantic segmentation head embedding. The face encoder converts the input feature vector into an embedding shaped $N_f \times 64D$. Then the embedding is inner product with itself in the decoder to obtain a symmetric matrix of $N_f \times N_f$, where each position of the matrix represents the probability that the corresponding two faces belong to the same feature instance. The above process can be expressed as follows:

$$z = MLP(x) \quad (7)$$

$$S = \sigma(z^T z) \quad (8)$$

where x is the input feature vector, σ is the Sigmoid function, and S is the predicted link score. The loss function is the binary cross-entropy loss. It can be expressed as:

$$L_{Ins} = \frac{1}{N} \sum_{ij} -A_{ij} \log(S_{ij}) - (1 - A_{ij}) \log(1 - S_{ij}) \quad (9)$$

where A_{ij} is 1 when nodes i and j belong to the same instance and 0 otherwise. S_{ij} is the probability of predicting that nodes i and j belong to the same instance. N is the total number of elements involved in the loss calculation. Because the adjacency matrix A is sparse, the number of positive and negative samples is unbalanced which makes the model difficult to train. Therefore, the instance grouping head is trained using the technique of negative sampling. For each model, we sample the same number of negative labels as the number of positive labels for loss calculation and backpropagation to improve the network's training stability.

Based on probabilistic embedding [49], the **relation prediction head** encodes the constituent faces of each feature instance into a multivariate Gaussian distribution. Compared with encoding each face as an independent vector, the multivariate distribution can more accurately capture the relationship between feature instances and the internal structure to more naturally model the topological relationship between features. Its input comprises the task-specific feature vector and semantic embedding vectors from the previous two tasks. Two MLPs are used to learn the mean and standard deviation of the face distribution and encode them as 16D vectors, respectively. The hidden vector of the face embedding is obtained by sampling the encoded multivariate Gaussian distribution of each face. Then, through the inner product decoder and the linear classifier, whether there is a topological relationship between the feature instances and the type of relationship is obtained. The above process can be expressed as follows:

$$\mu = MLP(x) \quad (10)$$

$$\zeta = MLP(x) \quad (11)$$

$$z = \mu + \epsilon \times \zeta \quad (12)$$

$$S = \sigma(z^T z) \quad (13)$$

where x is the input feature vector, σ is the Softmax function, ϵ is a random variable that conforms to the standard normal distribution, and S is the predicted link score shaped $N_f \times N_f \times N_{rel}$. N_{rel} is the number of topological relation types. μ and ζ are the mean and standard deviation of each face probability embedding, respectively, with dimension $N_f \times 16D$. The loss function of the relation prediction head consists of two parts: the cross-entropy loss function measures the deviation of the prediction result from the true label, and the KL divergence loss function measures how close the distribution encoded by each face is to the multivariate Gaussian distribution. It can be expressed as:

$$L_{Rel} = CE + KL \quad (14)$$

$$CE = -\frac{1}{N_{fe}^2} \sum_{ij} \sum_{c=1}^M y_{ijc} \log(p_{ijc}) \quad (15)$$

$$KL = -\frac{1}{2} \text{Mean}\{\text{Sum}[1 + 2\log(\zeta) - \mu^2 - \zeta^2]\} \quad (16)$$

where N_{fe} is the number of feature instances included in the model, p_{ijc} is the predicted probability that the relationship category between instances i and j is c , and M is the total number of relationship categories. y_{ijc} is a sign function equal to 1 when the relationship category between instances i and j is c and 0 for the rest. Sum function in the KL loss

computation sums the feature dimensions of the probabilistic embeddings, and Mean function averages the embeddings over the N_f faces.

The losses of the three task heads need to be fused and back-propagated to update and optimize the parameters of MFTReNet. The commonly used loss combination function is the arithmetic average. However, experiments found that the convergence speeds of the three task losses of MFTReNet differ and cause an imbalance in training, which in turn affects the overall performance of the model. Although this phenomenon can be mitigated to a certain extent by weighted averaging of the losses, the optimal weights are difficult to be obtained manually. Therefore, this paper uses Geometric Loss [44] to fuse the losses of the three tasks by using the geometric average instead of the arithmetic average to learn the three tasks equally. The total loss is formulated as follows:

$$L_{Total} = \sqrt[3]{L_{Seg} \times L_{Ins} \times L_{Rel}} \quad (17)$$

4. Experimental results and discussion

In this section, MFTReNet is trained on open-source datasets such as MFACAD++ and MFInstSeg to evaluate our model's performance objectively. In addition, MFTReNet is compared with the best-performing models in the extant literature on the MFTCAD dataset proposed in this paper.

We implemented MFTReNet based on PyTorch Geometric library (PyG) v2.5.1 and trained the model on a cloud server with Intel(R) Xeon (R) Platinum 8352 V CPU @ 2.10 GHz, 120G RAM, and RTX4090 GPU. The number of GNN blocks in each shared and task-specific encoder in the topological encoder is set to 1, and the Mish function [50] is used as the activation layer in MLPs. The number of sampling points for each 3D model UV grid is set to 10. The batch size is set to 64, and the initial learning rate is 0.001. The training is performed using the AdamW optimizer with a weight decay of 0.01, and the loss on the validation set is used as an indicator of early stopping in 100 epochs. In addition, Stochastic Weight Averaging [51] is used during the training process to improve the stability of the training process. The first 20 epochs are set to train with the initial learning rate, and then the learning rate decays to one-tenth of the initial with a cosine decay strategy. The model weights are sliding averaged to improve the model's generalization ability.

4.1. Experiment on MFCAD++ dataset

Colligan et al. [18] presented MFCAD++, which contains 59,655 model files in STEP format, and each face is labeled as one of 24 categories of machining features. MFCAD++ dataset is split into a training set, validation set, and test set in the ratio of 70/15/15 %, and the same division strategy is adopted in our experiment. Since this dataset only includes labels for semantic segmentation tasks, the same metrics of Accuracy (Acc) and mean Intersection Over Union (mIOU) are chosen to evaluate the model performance. They are defined as follows:

$$\text{Acc} = \frac{N_{fc}}{N_f} \quad (18)$$

$$\text{mIOU} = \frac{1}{C} \sum_{i=1}^C \frac{A_i \cap B_i}{A_i \cup B_i} \quad (19)$$

where N_f is the number of faces in the model, and N_{fc} is the face with the correct predicted feature category. C is the number of feature categories in the dataset, which is 24 in MFACD++. A_i is the set of faces with category i in the true labeling, and B_i is the set of faces with category i in the predicted labeling.

We conducted comparative experiments between MFTReNet and several of the most representative MFR networks on the MFACD++ dataset, including PointNet++ [52], DGCNN [53], Hierarchical CADNet

[18], and AAGNet [19]. The experimental results on the test set are shown in Table 3. The experimental results for the networks used for comparison are obtained from [18] and [19]. MFTRNet is trained only on the semantic segmentation head, and the results come from the mean and standard deviation calculated after ten experiments. The currently best-performing model on this dataset is AAGNet, and it can be seen that the proposed model in this paper performs equally well with it on the two metrics of the test set. Table 3 also lists the number of parameters for each model, and it can be seen that MFTRNet has the smallest number of parameters (0.36 M) when only the semantic segmentation head is enabled. It means that MFTRNet can perform semantic segmentation tasks with cutting-edge accuracy using less memory and computation.

4.2. Experiment on MFInstSeg dataset

Wu et al. [19] contribute a 3D model dataset containing labels, including face semantic and feature instances, which contains 62,495 STEP-format 3D models and 24 classes of machining features. The data is split into training, validation, and test sets in the ratio of 70/15/15 %. The F_1 Score is chosen as the evaluation metric for the instance segmentation task, as in the literature:

$$F_1 \text{ Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (20)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (21)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (22)$$

where TP represents the count of faces accurately assigned to an instance, whereas FP corresponds to the count of faces incorrectly assigned to an instance. FN denotes the count of faces erroneously unassigned.

The semantic segmentation head and instance grouping head of MFTRNet are trained on MFInstSeg with the same training parameter settings as before, and the training results are obtained as shown in Table 4, where the results of AAGNet and ASIN are from [19]. It can be seen that MFTRNet is more accurate and fluctuates less on the semantic segmentation and instance grouping tasks compared to the existing models. This is because the MFTRNet model employs an architecture that explicitly separates expertise and shared knowledge, avoiding the interference of harmful parameters between the two tasks, achieving a better balance, and improving the robustness of the model. The multi-task architecture of our model results in a slight increase in parameters compared to AAGNet, which employs a “Hard Parameter Sharing” strategy. However, our model remains notably more lightweight than the point cloud-based ASIN. As demonstrated in Section 4.3, the modest expansion in the number of parameters enhances the accuracy and robustness of MFTRNet in the face of more pronounced feature cross-over and intricate multi-task correlations.

4.3. Experiment on MFTRCAD dataset

We conducted experiments on the MFTRCAD dataset proposed in

Table 3
Semantic accuracy on MFCAD++.

Network	Semantic Segmentation		Number of parameters
	Acc(%)	mIOU(%)	
PointNet++	85.88	—	1.42 M
DGCNN	85.98	—	0.53 M
Hierarchical CADNet	97.37	—	9.76 M
AAGNet	99.26 ± 0.02	98.66 ± 0.02	0.38 M
MFTRNet	99.30 ± 0.01	98.63 ± 0.01	0.36 M

Table 4

Semantic segmentation and instance grouping performance on MFInstSeg.

Network	Semantic Segmentation		Instance Grouping		Number of parameters
	Acc(%)	mIOU(%)	Acc(%)	F_1 Score	
ASIN	86.46 ± 0.45	79.15 ± 0.82	98.29 ± 0.07	73.20 ± 0.86	6.08 M
AAGNet	99.15 ± 0.03	98.45 ± 0.04	99.94 ± 0.01	98.84 ± 0.07	0.41 M
MFTRNet	99.56 ± 0.02	98.43 ± 0.03	99.95 ± 0.01	98.90 ± 0.02	0.58 M

this paper to thoroughly evaluate the comprehensive performance of MFTRNet on the three tasks of semantic segmentation, instance grouping, and topological relationship prediction. The dataset consists of 28,661 3D models in STEP format, each with the instance-level semantic label of machining features and topological relationship label. The MFTRCAD dataset is randomly divided into a 70 % training set, a 15 % validation set, and a 15 % test set. A comprehensive comparison of MFTRNet with current state-of-the-art MFR methods is provided herein. This includes models with semantic segmentation only: PointNet++ [52], DGCNN [53], and Hierarchical CADNet [18], and also multi-task models with both semantic segmentation and instance grouping: ASIN [12] and AAGNet [19]. For the three multi-task models (ASIN, AAGNet, and MFTRNet), we also train each task head separately. This allows us to evaluate the difference in model performance on single and multi-tasks.

Experiments are conducted on MFTRCAD dataset, evaluating both the face-level and feature-level performance. The face-level evaluation metrics included the accuracy of the semantic segmentation task and the F_1 Score of the instance grouping task. The network is evaluated to determine if it correctly categorized each face of a part and clustered faces belonging to the same instance of a feature into a group.

In MFR, evaluating at the feature-level is more crucial than at the face-level since the planning of the machining process is based on feature instances rather than face entities. The feature-level evaluation metrics comprise the accuracy of feature recognition and the effectiveness of topological relationship prediction. Recognition accuracy at the feature-level involves recognizing and localizing feature instances by combining the prediction results of the two tasks of semantic segmentation and instance grouping. A feature instance is only considered a positive sample if each face it contains is correctly segmented and classified into the appropriate category. The topological relationship prediction task is a combination of link prediction and relationship classification, which needs to determine whether there is a relationship between each pair of feature instances and classify the possible relationships. Only a few features per part have topological relationships between them; thus, there is an imbalance in the labeling of the relationship prediction task. Therefore, we choose Average Precision (AP) as the evaluation metric to evaluate the model's performance in the relationship prediction task. The AP score is a weighted average of precision at each threshold and the difference between the recall and the previous threshold as the weight. It is defined as follows:

$$AP = \sum_n (R_n - R_{n-1}) P_n \quad (23)$$

where P_n, R_n is the respective precision and recall at threshold index n . This value is equivalent to the area under the precision-recall curve (AUPRC). AP combines precision and recall and does not ignore the less-sample category, making it suitable for evaluating relational prediction tasks.

We reproduced the individual comparison methods based on open-source code and performed multiple replicated experiments on the MFTRCAD dataset. More details are in Appendix B. For MFTRNet, the loss function is changed to Focal Loss, and the rest of the training settings are the same as in the previous section. The performance of these

models at the face-level and feature-level on the test set is shown in Table 5 and Table 6, respectively. All models are run ten times to calculate each metric's mean and standard deviation. The experimental results illustrate the superior performance of the proposed MFTRNet over the current state-of-the-art method, AAGNet. The MFTRNet achieves higher accuracy and less fluctuation in both face-level and feature-level recognition. At face-level, our model achieved an accuracy of $89.88 \pm 0.02\%$ for the semantic segmentation task and a F_1 Score of $85.35 \pm 0.03\%$ for the instance grouping task. Additionally, for the feature-level feature recognition and localization task, our model achieved an accuracy of $85.47 \pm 0.05\%$. At the same time, MFTRNet also shows an excellent result in relationship prediction tasks, with an AP of $98.10 \pm 0.01\%$.

In addition, we trained and evaluated ASIN, AAGNet and MFTRNet on a single task, and the results are also presented in Table 5 and Table 6. We calculated the Multi-Task Learning Gain (MTL Gain) of each task for the models, which is formulated as follows:

$$\Delta_m = \frac{(-1)^{l_t} (M_{m,t} - M_{b,t})}{M_{b,t}} \quad (24)$$

where l_t is an indicator that is 1 if a lower value of the metric M is more favorable, and 0 otherwise. $M_{m,t}$ is the performance metric of the multi-task learning model on task t , and $M_{b,t}$ is the performance metric of the single-task learning model on task t . The MTL Gain of the models on the MFTRCAD is shown in Fig. 9.

It can be seen that the MTL Gain of ASIN and AAGNet are negative on the two tasks of semantic segmentation and instance grouping, indicating that its multi-task model underperforms the single-task model, i. e., there is a performance degradation problem with negative transfer. This may be due to the fact that the feature crossover phenomenon is more severe in multi-layer part 3D models, resulting in complex correlations among the three tasks of MFR. The traditional "Hard Parameter Sharing" multi-task learning framework can lead to performance degradation due to the interference of harmful parameters among tasks. The multi-task learning approach adopted by MFTRNet, which explicitly separates specialized knowledge from public knowledge, can better solve this problem and improve the performance of the multi-task model compared to the single-task model.

To visually assess the effectiveness of MFTRNet for different tasks, we visualize the embeddings of the inputs of each task head. The embeddings output by the topological information encoder for each task are first extracted. Then, the embeddings are downsampled by Kernel Principal Component Analysis (KPCA), which visualizes them in three dimensions using the first, second, and third principal components after feature fusion. The visualization for the three tasks on a case model in

Table 5
Face-level performance on MFTRCAD test set.

Network	Semantic Segmentation Acc(%)	Instance Grouping F_1 Score(%)
PointNet++	67.89 $\pm$ 0.08	—
DGCNN	67.97 $\pm$ 0.07	—
Hierarchical CADNet	78.39 $\pm$ 0.03	—
ASIN-seg	68.57 $\pm$ 0.41	—
ASIN-ins	—	81.37 $\pm$ 0.09
ASIN-full	66.23 $\pm$ 0.76	72.55 $\pm$ 0.12
AAGNet-seg	79.45 $\pm$ 0.02	—
AAGNet-ins	—	82.44 $\pm$ 0.02
AAGNet-full	75.77 $\pm$ 0.03	74.98 $\pm$ 0.04
MFTRNet-seg	87.07 $\pm$ 0.03	—
MFTRNet-ins	—	81.33 $\pm$ 0.03
MFTRNet-full	89.88 $\pm$ 0.02	85.35 $\pm$ 0.03

*"model-seg" indicates that only the semantic segmentation head of this multi-task model is trained. "model-ins" indicates that only the instance grouping head of this multi-task model is trained. "model-full" represents training on both semantic segmentation and instance grouping tasks.

Table 6
Feature-level performance on MFTRCAD test set.

Network	Recognition & Localization Acc(%)	Relationship Prediction AP(%)
ASIN-full	60.43 $\pm$ 0.93	—
AAGNet-full	70.38 $\pm$ 0.15	—
MFTRNet-rel	—	92.48 $\pm$ 0.02
MFTRNet-full	85.47 $\pm$ 0.05	98.10 $\pm$ 0.01

*"model-rel" represents training only the topological relation prediction head of MFTRNet.

MFTRCAD is shown in Fig. 10. It can be observed that there are two 6-sided Passage features with a Mirror relationship in the model, and each feature consists of six faces. In the semantic segmentation feature space, these 12 faces are close together because they belong to the same feature type, and different types of features are farther apart. In the instance grouping feature space, two instances of 6-sided Passage features are farther apart. That is, different instances of the same type are farther apart, while instances of different types may instead be closer, thus effectively distinguishing different feature instances of the same type. In the relational prediction feature space, feature instances with topological relationships are pulled to a closer distance, thus realizing the semantic embedding of feature relationships.

4.4. Ablation experiments

This section examines the impact of various network structures on the model's performance. Four aspects of the network structure are considered: the multi-task parameter sharing architecture, the number of GNN blocks in the topological information encoder, the addition of semantic links between the multi-task heads, and the selection of the loss combination function. We change these parameters and evaluate the model performance on the MFTRCAD dataset. The results are shown in Table 7, and the numbers in parentheses are the decrease in the performance index after the ablation of the structure. Fig. 11 shows more visually the performance variations of the different architectures of MFTRNet.

First, we change the multi-task parameter-sharing architecture. We implement the "Hard Parameter Sharing" strategy used in AAGNet and ASIN. This involves retaining only the shared encoder in the topological information encoder of MFTRNet and inputting the embedding output from the shared encoder directly into the three task heads for prediction. In order to ensure the fairness of the comparison experiments, we double the feature embedding dimensions of the topological information encoder in the "Hard Parameter Sharing" architectural model. This ensures that the number of parameters of the models in the two multi-task architectures is similar. From the experimental results, it can be seen that the accuracy of the three tasks under the new architecture generally decreases. Even worse, the model performance after adjusting the parameter-sharing strategy is lower than the single-task learning accuracy shown in Table 5 and Table 6, i.e., a negative transfer problem occurs. This proves that explicitly separating shared and task-specific knowledge in multi-task learning effectively solves the negative transfer problem.

We then changed the number of GNN blocks in the shared encoder and task-specific encoder, with the number of GNN blocks in each encoder set to 2 and 3 for training, respectively. From the results, it can be seen that there is little difference in the performance of the models with a number of GNN blocks of 2 or 3, and both are significantly lower than the default setting of 1 GNN block. Meanwhile, during the training process, it is found that the training epoch decreased with an increased number of GNN blocks due to the settings' early stopping mechanism. It can be judged that increasing the number of GNN blocks leads to a sharp increase in the number of model parameters, which is easier to overfit under the current data volume. Therefore, the number of GNN blocks in

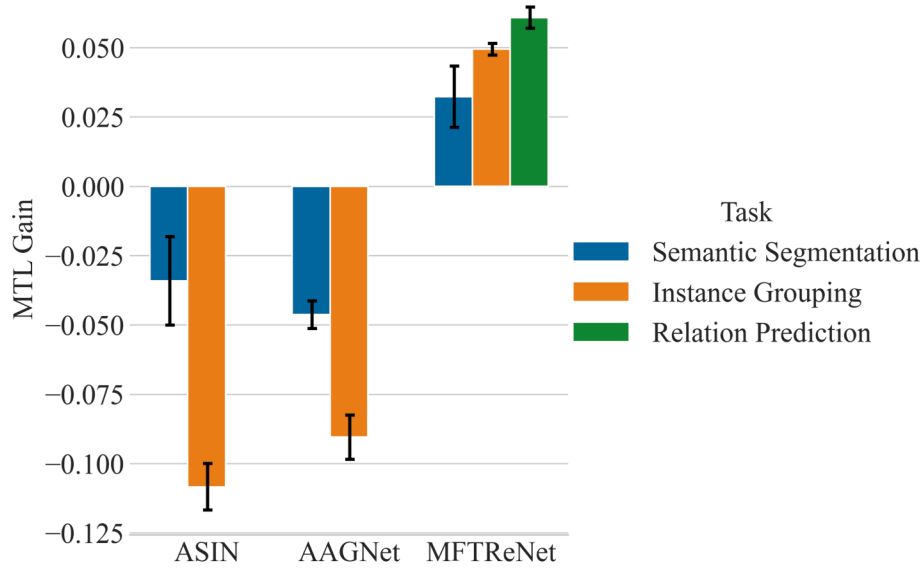


Fig. 9. MTL Gain of the multi-task models.

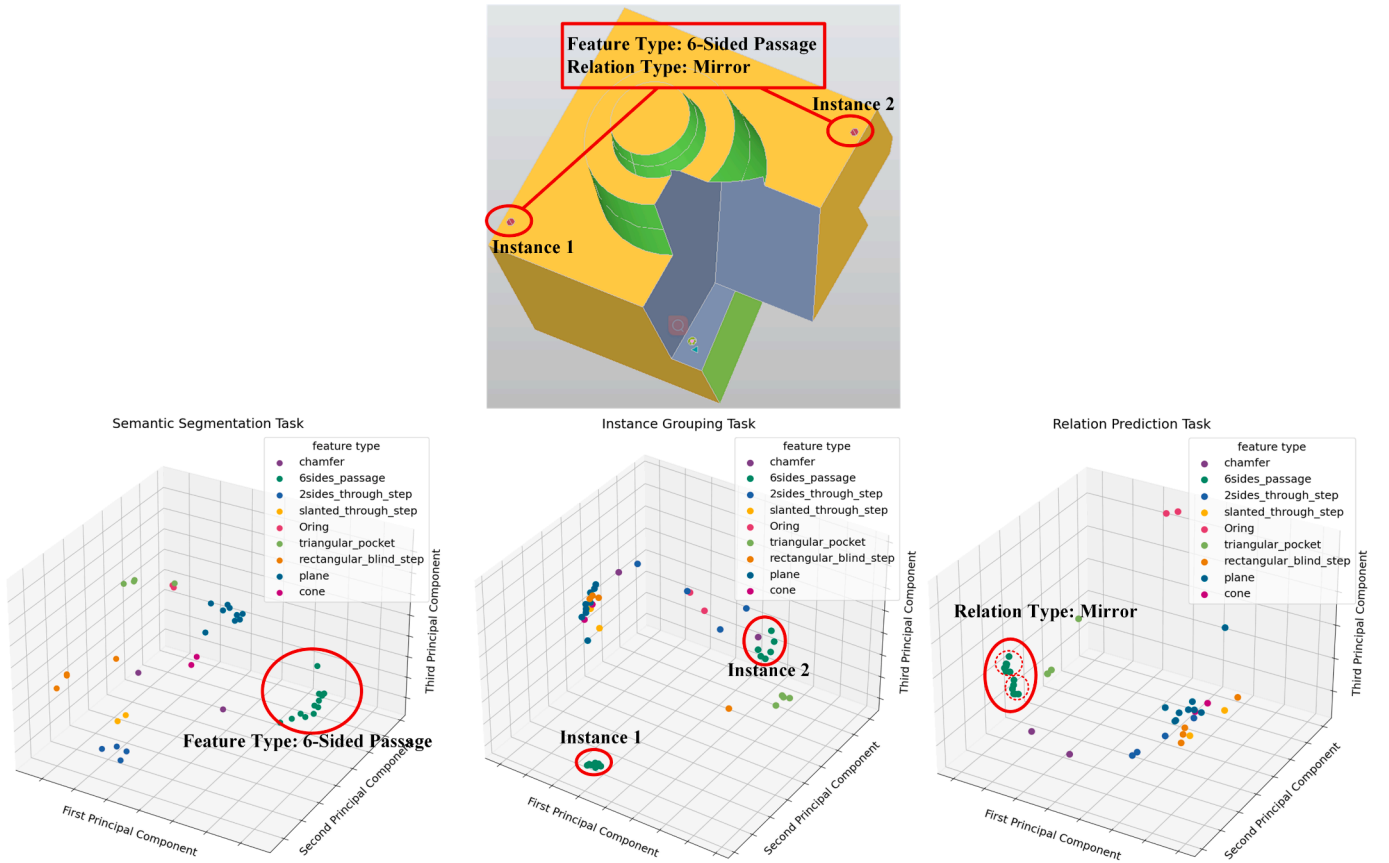


Fig. 10. Visual cases of three tasks of MFTRNet.

the model should be set flexibly according to the training data volume.

Next, we removed the semantic links between the three task heads and observed decreased model accuracy. The facilitating effect of semantic links on model learning effectiveness is demonstrated.

Finally, we change the Geometric Loss to an arithmetic average loss. It is evident that the model's performance significantly deteriorates across all three tasks. Fig. 12 shows the validation loss curves for the three task training phases using different combination functions. The

model using Geometric Loss demonstrates better convergence on all three tasks. Meanwhile, due to the inconsistent convergence rate of the loss on the three tasks, the arithmetic mean cannot weigh the gradients of multiple tasks and even shows a trend of increased loss in the later stages of training of the instance grouping task. The training is terminated early due to the early stopping mechanism. It can be concluded that the Geometric Loss balances the multi-task weights through geometric transformation, improving the stability of the model training

Table 7
The results of ablation experiments.

Network Architecture	Semantic Segmentation Acc(%)	Instance Grouping F_1 Score(%)	Relationship Prediction AP(%)
Default	89.88 ± 0.02	85.35 ± 0.03	98.10 ± 0.01
Hard Parameter Sharing	86.26 ± 0.05 (−3.62)	80.94 ± 0.04 (−4.41)	97.67 ± 0.02 (−0.43)
2 GNN Blocks	84.45 ± 0.02 (−5.43)	74.33 ± 0.03 (−11.02)	95.88 ± 0.02 (−2.22)
3 GNN Blocks	84.49 ± 0.02 (−5.39)	73.98 ± 0.02 (−11.37)	95.37 ± 0.01 (−2.73)
No Semantic Links	86.20 ± 0.03 (−3.68)	80.60 ± 0.04 (−4.75)	96.74 ± 0.02 (−1.36)
Use Arithmetic	77.96 ± 0.06 (−11.92)	80.85 ± 0.07 (−4.50)	96.37 ± 0.03 (−1.73)
Average Loss			

process and enhancing model performance.

4.5. Case study

This section validates the effectiveness of the proposed method in a real machining environment. MFTRNet's feature recognition, localization, and topological relationship extraction capabilities are verified through examples on three genuine part 3D models. The results, which list the feature instances, topological relationships, and recognition time by MFTRNet on each model, are shown in Fig. 13.

The three example parts consist of multiple layers that are

combinations of cubes and rotators. MFTRNet is capable of distinguishing machining feature faces from base faces based on topological information, thereby avoiding incorrect recognition of base faces as machining feature faces. In addition, MFTRNet can handle intersecting machining features, such as Ins.7 and Ins.8 in part B. Despite the geometrical discontinuities and topological changes caused by their intersection, MFTRNet can still accurately distinguish and localize the two feature instances of Ins.7 and Ins.8. MFTRNet has the unique ability to extract topological relationships that other methods lack. This feature improves the results of machining feature recognition and enables better application to intelligent process planning. For instance, in part A, three same-axis holes with a *Depend-on* relationship require a specific machining sequence to ensure coaxiality and precision. The correct sequence is to machine the large holes first, followed by the small holes. The machining process can be improved by arranging a *Mirror* or *Array* relationship of holes and slots. It is recommended that the machining process for rounds and chamfers be carried out immediately after the end of the *Transition* features.

In summary, MFTRNet accurately and efficiently segments each machining feature instance and predicts the topological relationship between instances. This is achieved by combining its machining feature recognition capability and topological relationship extraction capability, providing support for intelligent process planning.

5. Conclusion

In this paper, based on the graph neural network and multi-task

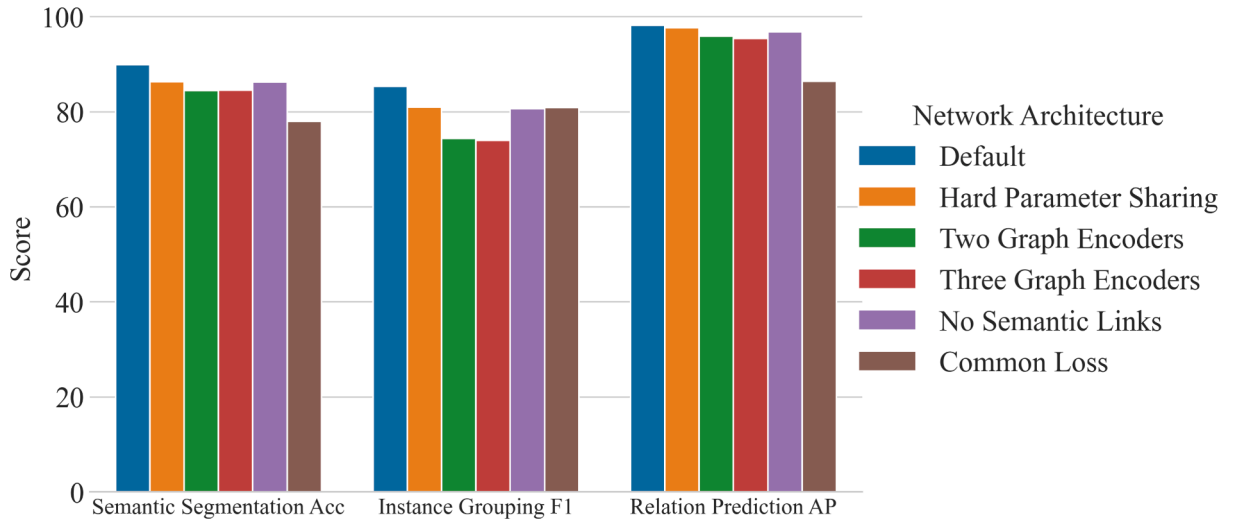


Fig. 11. Visualization of ablation experiment results.

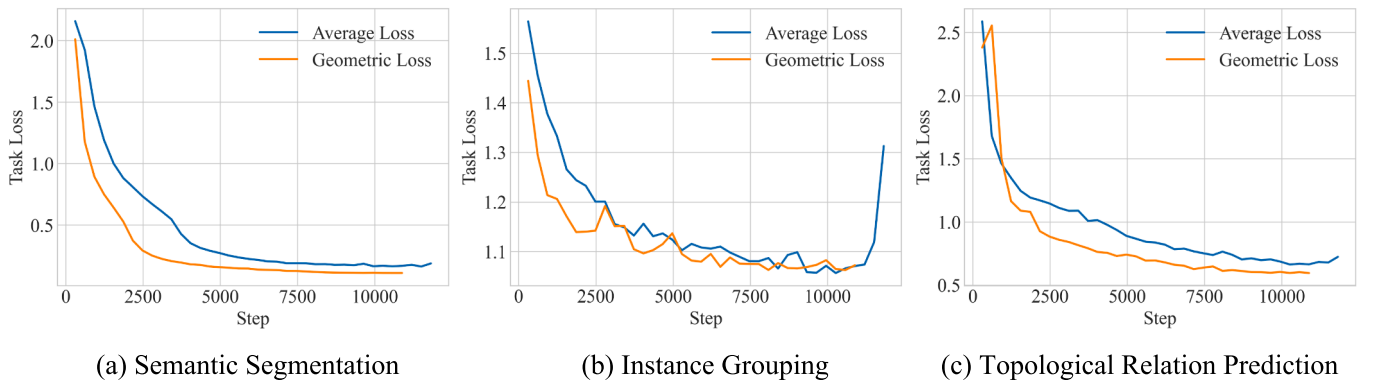


Fig. 12. Change of validation loss using different loss combination functions.

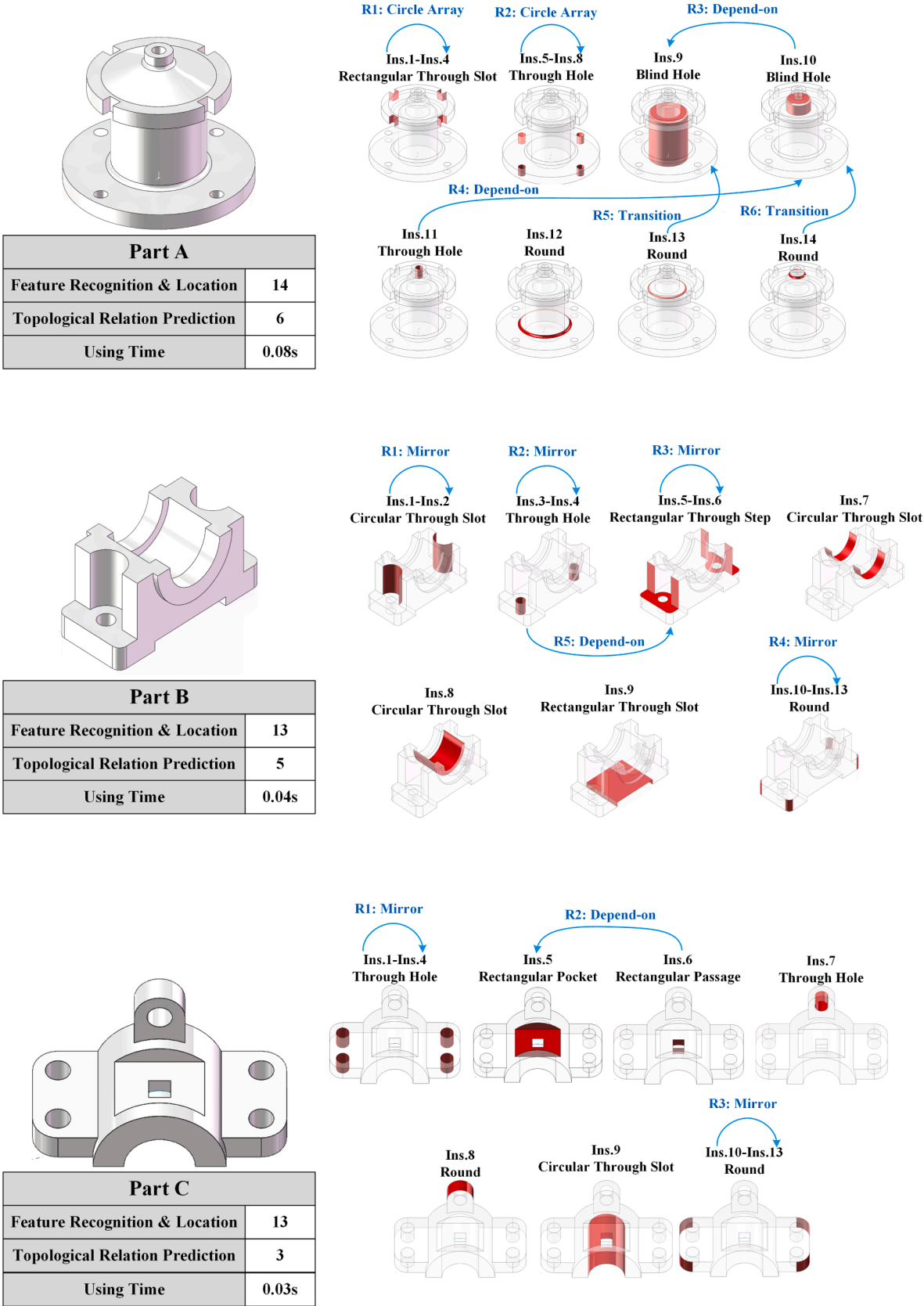


Fig. 13. Recognition result of genuine parts.

learning paradigm, a novel machining feature recognition method called MFTRNet is proposed, which not only realizes the recognition and localization of machining features but also predicts the topological relationships among features. To train MFTRNet, this paper also creates a dataset containing more than 20 k CAD models called MFTRCAD, which has models closer to real multi-layer parts and labels topological relationships between features. We have compared and evaluated MFTRNet with current cutting-edge MFR methods on various open-source datasets and demonstrated that MFTRNet has even better semantic segmentation and instance grouping accuracy. At the same time, MFTRNet has the ability to recognize topological relations that existing learning-based methods do not have. The automated extraction of machining features and topological relations is accomplished at the same time, which will facilitate the integration of data from the product life cycle and lay the foundation for more intelligent process planning.

However, there are still limitations and challenges that need to be addressed in future work. Below are some potential research directions:

- To realize intelligent process planning, it is necessary to investigate how to realize the mapping and correlation between process information, such as roughness and tolerance in the MBD model, and the extracted machining features.
- The multi-task learning architecture used in MFTRNet explicitly separates shared knowledge from task-specific knowledge. However, the lack of constraints between individual encoding modules may result in feature redundancy, which affects model performance.
- The dataset currently covers a limited number of feature types and relationship types, which restricts the application scenarios of the model, so the combination of unsupervised learning and heuristic

learning is subsequently considered to enhance the generalized capability of the model further.

The MFTRCAD is available at <https://www.kaggle.com/datasets/xmy2000/mftrcad>.

The MFTRNet is available at <https://github.com/xmy2000/MFTRNet>.

Funding

This work was supported by the Science and Technology Commission of Shanghai Municipality (Grant No. 23XD1450400).

CRediT authorship contribution statement

Mingyuan Xia: Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Xianwen Zhao:** Writing – review & editing, Conceptualization. **Xiaofeng Hu:** Writing – review & editing, Supervision, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

I have shared the link to my data and code in the manuscript.

Appendix A. . Dataset statistics

The dataset (MFTRCAD) proposed in this paper contains 28,661 3D models in STEP format. Each model consists of a multi-layer base composed of one or more cubes, cylinders, and cones, and 3–10 machining features are randomly applied to the base. The topological relationships among the features are also randomly established. Fig. 14 visualizes some of the samples in the dataset, with different features distinguished using colors. Fig. 15 and Fig. 16 show the number of samples in the dataset for each type of feature and topological relationship, respectively.

Each model in the dataset contains three types of labels. The format of semantic segmentation labels is $\{F_i : C_i\}$, where F_i is the face ID and C_i is the category of machining feature to which the face belongs. The format of instance grouping labels is $\{Fe_i : [F_j]\}$, where Fe_i is the feature instance ID and $[F_j]$ is the list of the instance grouping faces. The topological relationship label format is $\{R_i : [Fe_i]\}$, where R_i is the relationship type and $[Fe_i]$ is the list of feature instances that make up the relationship.



Fig. 14. Some models in the MFTRCAD

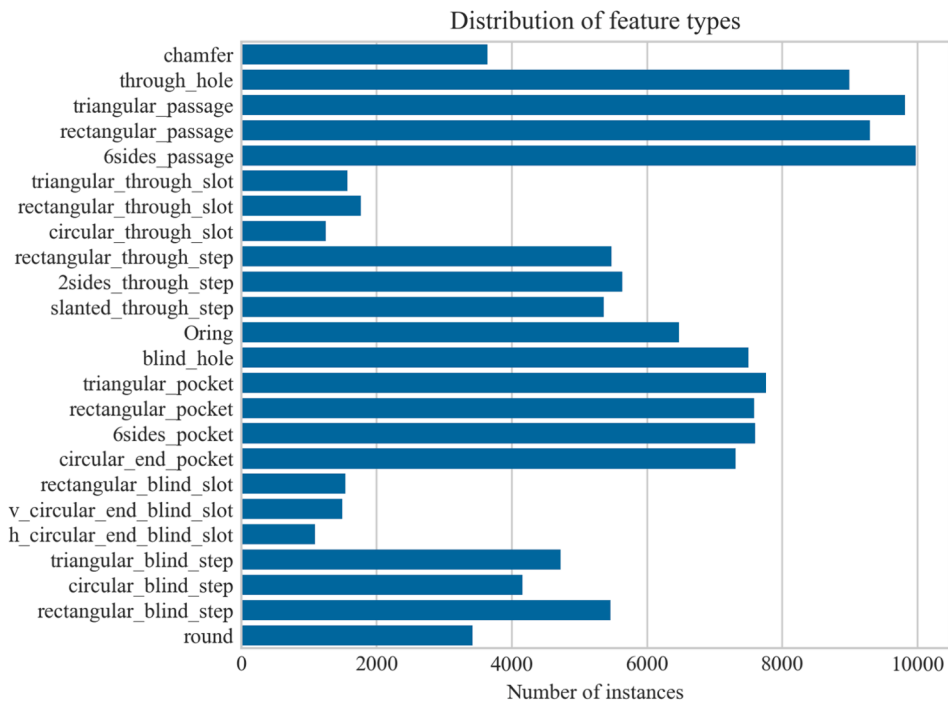


Fig. 15. Distribution of feature types in MFTRCAD

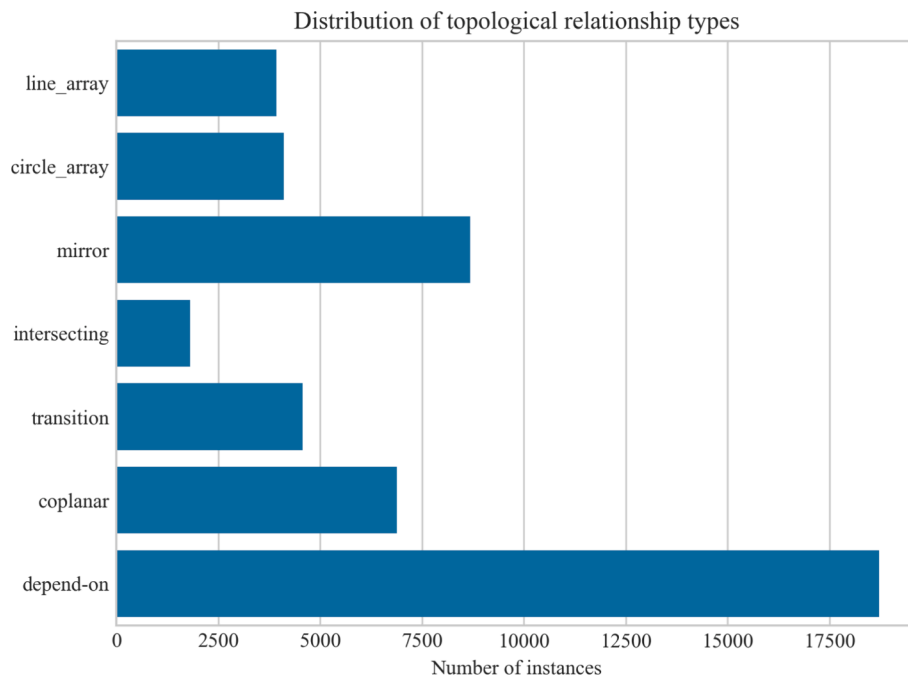


Fig. 16. Distribution of topological relationship types in MFTRCAD

Appendix B. . Referenced open-source code repositories

In order to conduct comparison experiments on the MFTRCAD dataset, we replicated each comparison method based on the following open-source code repository. After verifying that the reproduced individual models can achieve the performance in the original papers, they are then used on the MFTRCAD dataset for training and testing.

- PointNet++ implementation: https://github.com/yanx27/Pointnet_Pointnet2_pytorch
- DGCNN implementation: <https://github.com/AnTao97/dgcnn.pytorch>
- Hierarchical CADNet implementation: https://gitlab.com/qub_femg/machine-learning/hierarchical-cadnet
- ASIN implementation: <https://github.com/HARRIXJANG/ASIN-master>
- AAGNet implementation: <https://github.com/whjdark/AAGNet>

References

- [1] J. Zhou, P. Li, Y. Zhou, B. Wang, J. Zang, L. Meng, Toward new-generation intelligent manufacturing, *Engineering* 4 (2018) 11–20.
- [2] G. Colombo, A. Mosca, F. Sartori, Towards the design of intelligent CAD systems: an ontological approach, *Adv. Eng. Inf.* 21 (2007) 153–168, <https://doi.org/10.1016/j.aei.2006.11.003>.
- [3] C. Favi, F. Campi, M. Germani, M. Mandolini, Engineering knowledge formalization and proposition for informatics development towards a CAD-integrated DfX system for product design, *Adv. Eng. Inf.* 51 (2022) 101537, <https://doi.org/10.1016/j.aei.2022.101537>.
- [4] Y. Shi, Y. Zhang, K. Xia, R. Harik, A critical review of feature recognition techniques, *CADandA* 17 (2020) 861–899, <https://doi.org/10.14733/cadaps.2020.861-899>.
- [5] T. Yifei, L. Dongbo, L. Changbo, Y. Minjian, A feature-extraction-based process-planning system, *Int. J. Adv. Manuf. Technol.* 38 (2008) 1192–1200.
- [6] A. McCormack, R.N. Ibrahim, Process planning using adjacency-based feature extraction, *Int. J. Adv. Manuf. Technol.* 20 (2002) 817–823.
- [7] S. Ding, Q. Feng, Z. Sun, F. Ma, MBD based 3D CAD model automatic feature recognition and similarity evaluation, *IEEE Access* 9 (2021) 150403–150425.
- [8] B. Babic, N. Nesic, Z. Miljkovic, A review of automated feature recognition with rule-based pattern recognition, *Comput Ind* 59 (2008) 321–337, <https://doi.org/10.1016/j.compind.2007.09.001>.
- [9] A.K. Verma, S. Rajotia, A review of machining feature recognition methodologies, *Int. J. Comput. Integr. Manuf.* 23 (2010) 353–368.
- [10] M. Al-wswasi, A. Ivanov, A novel and smart interactive feature recognition system for rotational parts using a STEP file, *Int. J. Adv. Manuf. Technol.* 104 (2019) 261–284.
- [11] T. Xu, J. Li, Z. Chen, Automatic machining feature recognition based on MBD and process semantics, *Comput. Ind.* 142 (2022) 103736.
- [12] H. Zhang, S. Zhang, Y. Zhang, J. Liang, Z. Wang, Machining feature recognition based on a novel multi-task deep learning network, *Rob. Comput. Integr. Manuf.* 77 (2022) 102369.
- [13] Z. Zhang, P. Jaiswal, R. Rai, FeatureNet: machining feature recognition based on 3d convolution neural network, *Comput. Aided Des.* 101 (2018) 12–22.
- [14] A.C. Lin, S.-Y. Lin, D. Diganta, W.F. Lu, An integrated approach to determining the sequence of machining operations for prismatic parts with interacting features, *J. Mater. Process. Technol.* 73 (1998) 234–250.
- [15] D. Zhou, X. Dai, A method for discovering typical process sequence using granular computing and similarity algorithm based on part features, *Int. J. Adv. Manuf. Technol.* 78 (2015) 1781–1793.
- [16] Y. Xiong, A.G. Dharmawan, Y. Tang, S. Foong, G.S. Soh, D.W. Rosen, A knowledge-based process planning framework for wire arc additive manufacturing, *Adv. Eng. Inf.* 45 (2020) 101135, <https://doi.org/10.1016/j.aei.2020.101135>.
- [17] W. Cao, T. Robinson, Y. Hua, F. Boussuge, A.R. Colligan, W. Pan, Graph representation of 3D CAD models for machining feature recognition with deep learning, in: *International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, American Society of Mechanical Engineers, 2020: p. V11AT11A003.
- [18] A.R. Colligan, T.T. Robinson, D.C. Nolan, Y. Hua, W. Cao, Hierarchical cadnet: learning from b-reps for machining feature recognition, *Comput. Aided Des.* 147 (2022) 103226.
- [19] H. Wu, R. Lei, Y. Peng, L. Gao, AAGNet: a graph neural network towards multi-task machining feature recognition, *Rob. Comput. Integr. Manuf.* 86 (2024) 102661.
- [20] S. Ren, K. He, R. Girshick, J. Sun, Faster r-cnn: towards real-time object detection with region proposal networks, *Adv. Neural Inf. Process. Syst.* 28 (2015).
- [21] S. Hao, Y. Zhou, Y. Guo, A brief survey on semantic segmentation with deep learning, *Neurocomputing* 406 (2020) 302–321.

- [22] D. Bolya, C. Zhou, F. Xiao, Y.J. Lee, Yolact: Real-time instance segmentation, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2019: pp. 9157–9166.
- [23] X. Kong, W. Zhang, Y. Qu, X. Yao, G. Shen, FedAWR: an interactive federated active learning framework for air writing recognition, *IEEE Trans. Mob. Comput.* (2023).
- [24] X. Kong, K. Wang, M. Hou, X. Hao, G. Shen, X. Chen, F. Xia, A federated learning-based license plate recognition scheme for 5G-enabled internet of vehicles, *IEEE Trans. Ind. Inf.* 17 (2021) 8523–8530.
- [25] C. Gupta, N.S. Gill, P. Gulia, S. Yadav, G. Pau, M. Alibakhshikenari, X. Kong, A real-time 3-dimensional object detection based human action recognition model, *IEEE Open J. Comput. Soc.* 5 (2024) 14–26, <https://doi.org/10.1109/OJCS.2023.3334528>.
- [26] P. Shi, Q. Qi, Y. Qin, P.J. Scott, X. Jiang, Intersecting machining feature localization and recognition via single shot multibox detector, *IEEE Trans. Ind. Inf.* 17 (2020) 3292–3302.
- [27] P. Shi, Q. Qi, Y. Qin, P.J. Scott, X. Jiang, A novel learning-based feature recognition method using multiple sectional view representation, *J. Intell. Manuf.* 31 (2020) 1291–1309.
- [28] P. Wang, W. Liu, Y. You, A hybrid framework for manufacturing feature recognition from CAD models of 3-axis milling parts, *Adv. Eng. Inf.* 57 (2023) 102073.
- [29] P. Wang, W.-A. Yang, Y. You, A hybrid learning framework for manufacturing feature recognition using graph neural networks, *J. Manuf. Process.* 85 (2023) 387–404.
- [30] H. Zhang, W. Wang, S. Zhang, Y. Zhang, J. Zhou, Z. Wang, B. Huang, R. Huang, A novel method for intersecting machining feature segmentation via deep reinforcement learning, *Adv. Eng. Inf.* 59 (2024) 102256.
- [31] X. Yao, D. Wang, T. Yu, C. Luan, J. Fu, A machining feature recognition approach based on hierarchical neural network for multi-feature point cloud models, *J. Intell. Manuf.* 34 (2023) 2599–2610.
- [32] J.-L. Jia, S.-W. Zhang, Y.-R. Cao, X.-L. Qi, WeZhu, Machining feature recognition method based on improved mesh neural network, *Iranian J. Sci. Tech., Tran. Mech. Eng.* 47 (2023) 2045–2058.
- [33] P. Shi, Q. Qi, Y. Qin, P.J. Scott, X. Jiang, Highly interacting machining feature recognition via small sample learning, *Rob. Comput. Integr. Manuf.* 73 (2022) 102260.
- [34] C. Yeo, B.C. Kim, S. Cheon, J. Lee, D. Mun, Machining feature recognition based on deep neural networks to support tight integration with 3D CAD systems, *Sci. Rep.* 11 (2021) 22147.
- [35] V. Miles, S. Giani, O. Vogt, Recursive encoder network for the automatic analysis of STEP files, *J. Intell. Manuf.* 34 (2023) 181–196.
- [36] V. Miles, S. Giani, O. Vogt, Approaching STEP file analysis as a language processing task: a robust and scale-invariant solution for machining feature recognition, *J. Comput. Appl. Math.* 427 (2023) 115166.
- [37] P.K. Jayaraman, A. Sanghi, J.G. Lambourne, K.D. Willis, T. Davies, H. Shayani, N. Morris, Uv-net: Learning from boundary representations, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2021: pp. 11703–11712.
- [38] T. Paviot, pythonocc, (2022). doi: 10.5281/zenodo.7471333.
- [39] M. Fey, J.E. Lenssen, Fast graph representation learning with PyTorch geometric, in: ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019.
- [40] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, S.Y. Philip, A comprehensive survey on graph neural networks, *IEEE Trans. Neural Networks Learn. Syst.* 32 (2020) 4–24.
- [41] L. Wu, P. Cui, J. Pei, L. Zhao, X. Guo, Graph neural networks: foundation, frontiers and applications, in: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2022: pp. 4840–4841.
- [42] R. Caruana, Multitask learning, *Mach. Learn.* 28 (1997) 41–75.
- [43] H. Tang, J. Liu, M. Zhao, X. Gong, Progressive layered extraction (ple): A novel multi-task learning (mtl) model for personalized recommendations, in: Proceedings of the 14th ACM Conference on Recommender Systems, 2020: pp. 269–278.
- [44] X. Bresson, T. Laurent, Residual gated graph convnets, *arXiv Preprint arXiv:1711.07553* (2017).
- [45] G. Corso, L. Cavalleri, D. Beaini, P. Liò, P. Velicković, Principal neighbourhood aggregation for graph nets, *Adv. Neural Inf. Proces. Syst.* 33 (2020) 13260–13271.
- [46] S. Brody, U. Alon, E. Yahav, How attentive are graph attention networks?, *arXiv Preprint arXiv:2105.14491* (2021).
- [47] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, *arXiv Preprint arXiv:1609.02907* (2016).
- [48] T.-Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár, Focal loss for dense object detection, in: Proceedings of the IEEE International Conference on Computer Vision, 2017: pp. 2980–2988.
- [49] B. Zhang, P. Wonka, Point cloud instance segmentation using probabilistic embeddings, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2021: pp. 8883–8892.
- [50] D. Misra, Mish: A self regularized non-monotonic activation function, *arXiv Preprint arXiv:1908.08681* (2019).
- [51] P. Izmailov, D. Podoprikin, T. Garipov, D. Vetrov, A.G. Wilson, Averaging weights leads to wider optima and better generalization, *arXiv Preprint arXiv:1803.05407* (2018).
- [52] C.R. Qi, L. Yi, H. Su, L.J. Guibas, Pointnet++: deep hierarchical feature learning on point sets in a metric space, *Adv. Neural Inf. Proces. Syst.* 30 (2017).
- [53] Y. Wang, Y. Sun, Z. Liu, S.E. Sarma, M.M. Bronstein, J.M. Solomon, Dynamic graph cnn for learning on point clouds, *ACM Tran. Graph. (Tog)* 38 (2019) 1–12.